

Contents

1	Introduction	3
1.1	What this course will teach	3
1.2	Project Kit Contents	3
1.3	Working with the microcontroller	3
1.3.1	Installing the driver	3
1.3.2	Pinout Reference	3
1.3.3	Project Guide Updates	4
2	Electronics Essentials	5
2.1	The Breadboard: Your Circuit's Foundation	5
2.1.1	What is a breadboard?	5
2.1.2	Understanding the layout	5
2.2	LEDs: Lighting Up Your Circuits	6
2.2.1	What is an LED?	6
2.2.2	Polarity	6
2.2.3	Current-limiting resistor	6
2.3	Resistors: Controlling Current Flow	7
2.3.1	What is a resistor?	7
2.3.2	Resistance values and identification	7
2.4	Buttons: User Input	7
2.4.1	What is a pushbutton?	7
2.4.2	Understanding the 4-pin connection	7
2.4.3	Internal pull-up resistors	7
2.5	Jumper Wires: Making Connections	8
2.5.1	What are jumper wires?	8
2.5.2	Wire colors	8
2.6	Safety Guidelines	8
3	MicroPython IDE	9
3.1	Connecting Your Microcontroller	9
4	Project 1: Night Light	11
4.1	Overview	11
4.2	Components	12
4.3	Directions	12
4.3.1	Creating the circuit	12
4.3.2	Programming the microcontroller	15
4.3.3	How the program works	15
4.4	Review	16
4.5	Possible Extensions	16
5	Project 2: Red Light, Green Light	17
5.1	Overview	17
5.2	Components	18
5.3	Directions	18
5.3.1	Creating the circuit	18
5.3.2	Programming the microcontroller	19
5.3.3	Hardware test checklist	20
5.4	Review	20
5.5	Possible Extensions	20

6	Project 3: Safe Cracker	21
6.1	Overview	21
6.2	Components	22
6.3	Directions	22
6.3.1	Understanding the KY-040 Rotary Encoder	22
6.3.2	Creating the circuit	23
6.3.3	Understanding the Code	24
6.3.4	Programming the microcontroller	26
6.4	Review	26
6.5	Possible Extensions	26
7	Project 4: Pocket DJ	27
7.1	Overview	27
7.2	Components	28
7.3	Directions	28
7.3.1	Creating the circuit	28
7.3.2	Programming the microcontroller	30
7.3.3	How the program works	31
7.3.4	Hardware test checklist	31
7.4	Review	31
7.5	Possible Extensions	32
8	Project 5: Wi-Fi Mood Lamp	33
8.1	Overview	33
8.2	Components	34
8.3	Directions	34
8.3.1	Creating the circuit	34
8.3.2	Programming the microcontroller	35
8.4	Review	36
8.5	Possible Extensions	36
9	Project 6: ESP-NOW Hot Potato	37
9.1	Overview	37
9.2	Directions	38
9.2.1	Creating the circuit	38
9.2.2	Programming the microcontroller	38
9.3	Review	38
9.4	Possible Extensions	38
10	Project 7: Tilting Ball Maze	39
10.1	Overview	39
10.2	Components	40
10.3	Understanding I ² C Communication	40
10.4	Directions	40
10.4.1	Creating the circuit	40
10.4.2	Programming the microcontroller	42
10.4.3	How the program works	42
10.5	Review	43
10.6	Possible Extensions	44
A	Python Primer	45
A.1	Part 1: General Python Fundamentals	45
A.1.1	A simple Python script	45
A.1.2	Breaking down the syntax	45
A.2	Part 2: MicroPython on Hardware	46
A.2.1	The microcontroller program anatomy	46
A.2.2	Key hardware programming patterns	46
A.2.3	Object-Oriented Programming (Classes) in MicroPython	47
A.3	MicroPython Concepts Reference Map	47

Chapter 1: Introduction

1.1 What this course will teach

This workshop will introduce the student to Python coding, electronics, and project design. We will be building several projects ranging from simple (less components and simpler code) to more complex (more components or more complicated code). These projects are based on the ESP32-C3 microcontroller (specifically the development board produced by Seeed Studio) which is running MicroPython and they depend on some other electronics components such as LEDs, buttons, and more.

Students are encouraged to make use of the written instructions and diagrams and to explore and play with the components for themselves to see how they work and what they do. There are many online resources as well for learning.

New to Python or MicroPython? If Python is new to you, or if you would like a quick refresher on how MicroPython interacts with hardware pins, check out **Appendix A (Python Primer)** before starting Project 1. If you already have Python experience, the primer's MicroPython section provides a handy overview of the hardware patterns used across all the projects.

1.2 Project Kit Contents

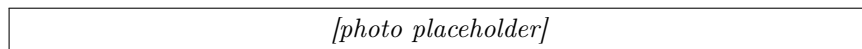


Figure 1.1: This image shows all of the contents of the project kit and labels what each one is

1.3 Working with the microcontroller

1.3.1 Installing the driver

Connecting the microcontroller to your computer may require the installation of a USB driver. If so, download the driver for your OS from this page and install as instructed: <https://ftdichip.com/drivers/vcp-drivers/>. Once installed successfully, unplug and replug the USB cable into the microcontroller and then press the small reset button on the microcontroller board labeled "R" (for reset).

1.3.2 Pinout Reference

On any integrated circuit, that is a chip that contains electronic logic, a pinout is a diagram and description of what each of the pins of that circuit are used for. This is a very useful document for any electronics engineer to have handy and to reference. The pinout diagram for the microcontroller that we are using as part of this workshop is replicated below:

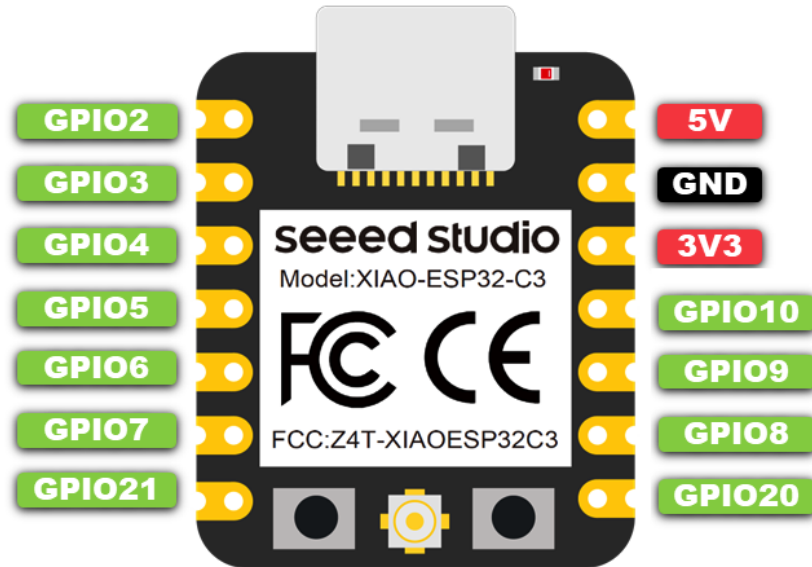


Figure 1.2: A simplified pinout of the Sseed Studio ESP32-C3 development board

In this diagram, you'll notice several things:

- An image of the microcontroller board in the center
- A label for each pin indicating its function

In the code throughout this guide, you will see references to pin numbers like **Pin(5)**. Wherever you see this, it means that the code is expecting a connection to, in this example, the 4th pin down on the left. If you find that your code is not working as expected, double check the pin numbers in the code vs. the placement of your wires.

1.3.3 Project Guide Updates

In case you are reading an out of date version of this guide, you can find the latest at https://netapp-ptc.github.io/YWIT-2026/project_guide.pdf.

You can also find the source code for all of the workshops and other useful information in the GitHub repository at <https://github.com/NetApp-PTC/YWIT-2026>.

Chapter 2: Electronics Essentials

This beginner's guide introduces the essential components and circuit-building concepts you will use throughout this workshop. We will cover the breadboard, LEDs, current-limiting resistors, pushbuttons, jumper wires, and fundamental electrical safety. Later projects will introduce additional modules—such as addressable RGB LEDs, rotary encoders, speakers, OLED displays, and motion sensors—which connect using these same breadboard rows and power rails.

2.1 The Breadboard: Your Circuit's Foundation

2.1.1 What is a breadboard?

A breadboard is a reusable platform for prototyping electronic circuits without soldering. It contains a grid of spring-clip holes into which component leads and jumper wires are inserted to make electrical connections.

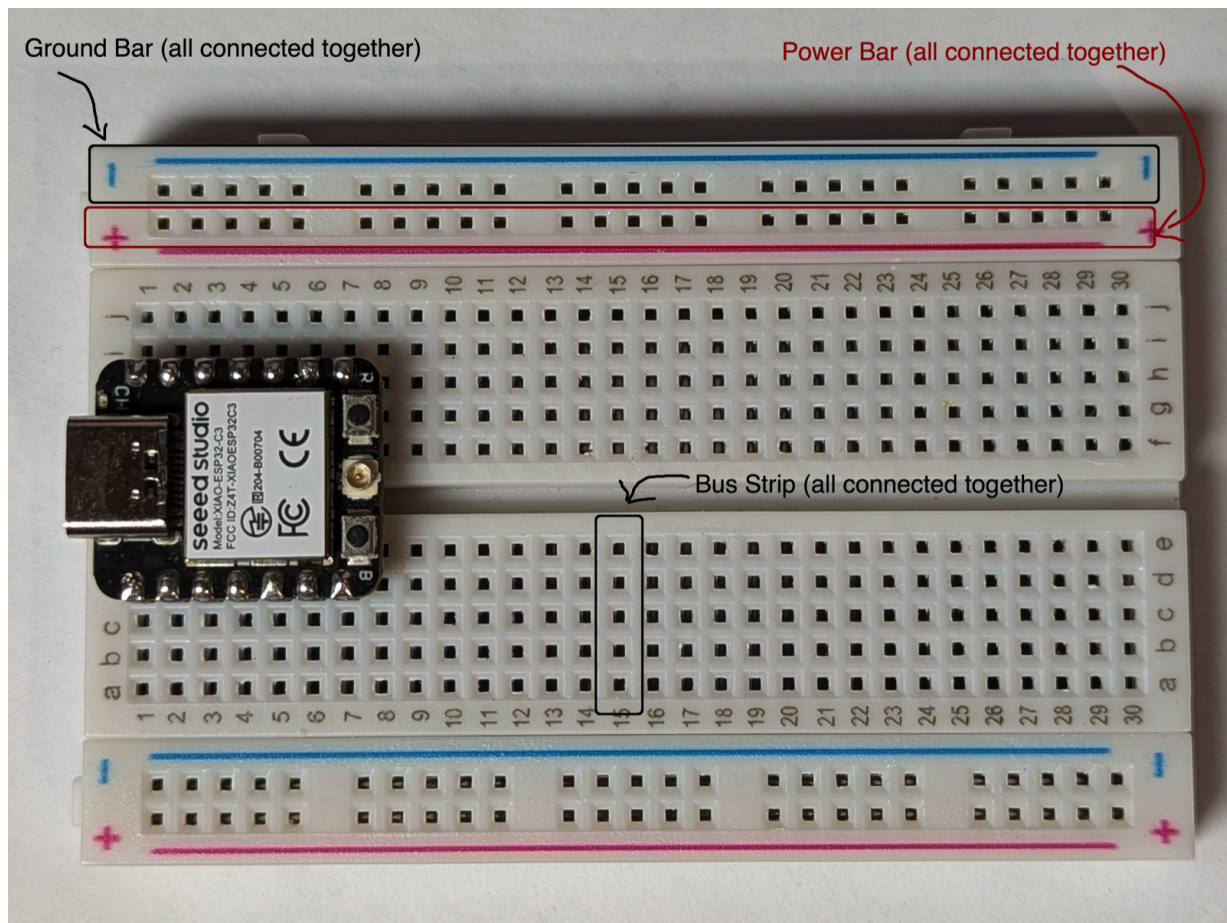


Figure 2.1: The workshop breadboard with the Seeed XIAO ESP32-C3 microcontroller seated.

2.1.2 Understanding the layout

Numbered columns (Bus Strips)

The main area of the breadboard is divided into lettered rows (labeled A through J) and numbered columns (labeled 1 to 30):

- On the bottom half, holes **A, B, C, D, and E** in the **same numbered column** are connected together internally by a vertical metal clip (a *bus strip*). For example, placing two component leads in **A10** and **C10** connects them together because both share column 10.
- On the top half, holes **F, G, H, I, and J** in the **same numbered column** are likewise connected together.
- **Adjacent column numbers are NOT connected.** Column 10 is completely separate from Column 11.

The center divider

A plastic trench separates rows A–E from rows F–J. The two sides are **electrically isolated** unless you connect a jumper wire across the gap. The microcontroller straddles this center trench so that its bottom pins (in row D) and top pins (in row H) connect to separate 5-hole bus strips.

Power rails

Along the top and bottom edges run long horizontal strips called **power rails**:

- **Positive rail (+, red line):** Used to distribute power (3.3V) across the board.
- **Ground rail (–, blue line):** Used as the common ground (GND) return path.

All holes along the same rail are connected horizontally across all columns. Once your microcontroller is seated, you will run jumper wires from its **3V3** and **GND** pins to these rails to power your circuits.

Breadboard summary: Holes in the **same numbered column** on the **same side** of the center divider (e.g., A10 through E10) are connected together. Holes in different numbered columns, or on opposite sides of the center divider, are separated until you connect them with a component or jumper wire.

2.2 LEDs: Lighting Up Your Circuits

2.2.1 What is an LED?

An LED (Light-Emitting Diode) is a semiconductor device that produces light when electric current flows through it in the correct direction.

2.2.2 Polarity

LEDs are **polarized**—electricity can only flow through them in one direction:

- **Anode (+):** The longer leg. Connects toward the positive voltage source (microcontroller GPIO pin).
- **Cathode (–):** The shorter leg. Connects toward Ground (GND). If the legs have been trimmed, look for the **flat edge** on the plastic rim of the LED casing on the cathode side.

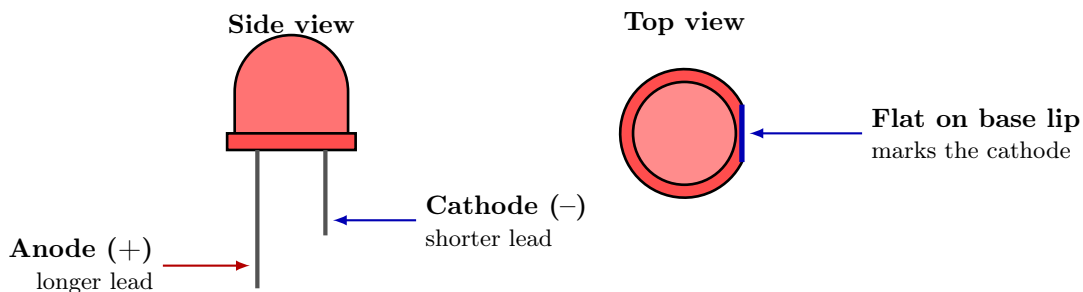


Figure 2.2: Use either clue to identify LED polarity: the cathode has the shorter lead and the flat edge.

2.2.3 Current-limiting resistor

LEDs have very little internal resistance. If connected directly between power and ground, too much current will flow, destroying the LED and potentially damaging the microcontroller pin.

You must **always connect a resistor in series** with an LED. “In series” means the current must travel through one single path: from the pin, through the LED, through the resistor, and into ground (or vice versa). In this kit, we use a **220Ω** resistor for the LED.

2.3 Resistors: Controlling Current Flow

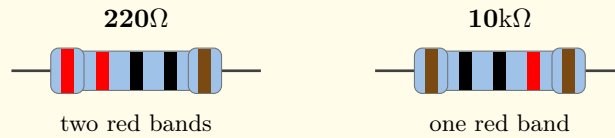
2.3.1 What is a resistor?

A resistor is a passive component that resists and limits the flow of electric current. Resistors protect sensitive components (like LEDs) and help create reference voltages for analog sensors. Resistors have **no polarity**—they can be installed in either orientation without affecting the circuit.

2.3.2 Resistance values and identification

Resistance is measured in ohms (Ω) and kilohms ($1\text{ k}\Omega = 1000\text{ }\Omega$). The resistance value is indicated by colored bands around the body of the resistor.

Which resistor is which? Both resistors are small and blue with five evenly spaced colored bands, and nothing is printed on them. You do not need to figure out which end to read from—just count the **red** bands. The **220 Ω** resistor has **two red bands next to each other**, and the **10k Ω** resistor has **exactly one red band**. Swapping the two will not damage anything, but the circuit will not behave the way this project describes, so ask an instructor if you are not sure.



2.4 Buttons: User Input

2.4.1 What is a pushbutton?

A pushbutton is a momentary switch that completes a connection when pressed and breaks it when released. The buttons in your kit have four pins arranged in two connected pairs.

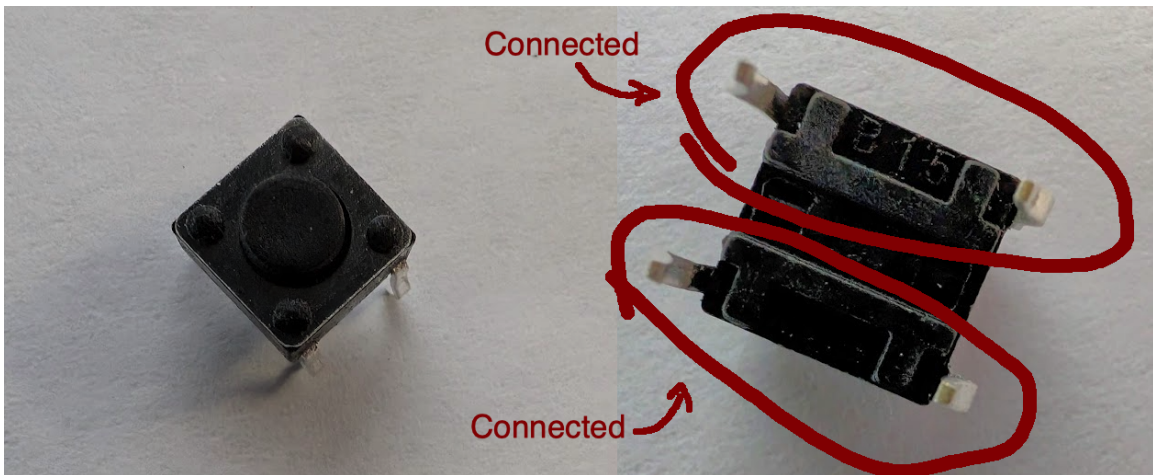


Figure 2.3: Pushbutton pin arrangement: the two pins on each side are internally joined.

2.4.2 Understanding the 4-pin connection

As shown in Figure 2.3, the two pins on the top edge are permanently joined internally, and the two pins on the bottom edge are permanently joined internally. Pressing the button bridges the gap between the top pair and the bottom pair. When placing the button across rows on the breadboard, ensure your two circuit connections go to opposite pairs so pressing the button closes the circuit.

2.4.3 Internal pull-up resistors

When a button is not pressed, an input pin would otherwise be left “floating” (unconnected), picking up electrical noise. The ESP32-C3 microcontroller has built-in **pull-up resistors** that can be enabled in MicroPython code. This pulls the pin

HIGH (3.3V) when idle; pressing the button connects the pin to GND, pulling it LOW (0V). This means you do not need an external resistor to wire a button!

Although both components are called resistors, an internal pull-up cannot replace the 220Ω current-limiting resistor used with an LED. A pull-up is a weak, relatively high-value resistor (typically tens of kilohms) connected inside the microcontroller between 3.3V and an **input** pin. Its job is only to keep an otherwise unconnected input at a known logic level. It is not in the LED's current path, its exact value is not tightly controlled, and it would supply far too little current to light the LED normally. The external 220Ω resistor has a known value and is physically placed **in series** with the LED so that all LED current passes through it.

2.5 Jumper Wires: Making Connections

2.5.1 What are jumper wires?

Jumper wires are flexible insulated wires with male pins at both ends that plug directly into breadboard holes.

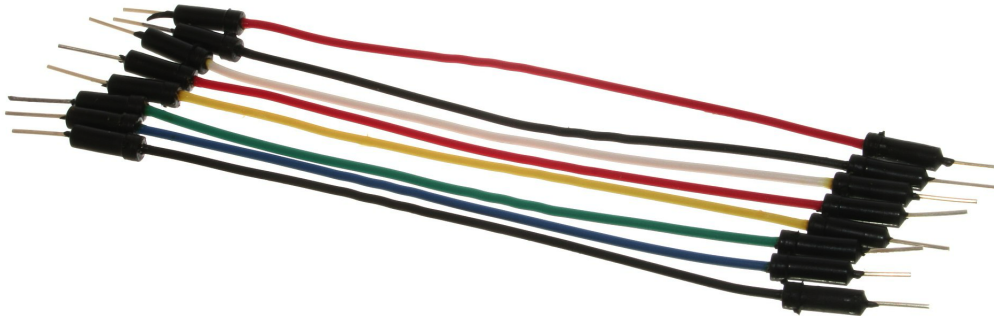


Figure 2.4: Male-to-male jumper wires in various colors.

2.5.2 Wire colors

All jumper wires are electrically identical on the inside. However, following standard color conventions makes your circuits much easier to read and troubleshoot:

- **Red / Orange:** Positive power supply (+3.3V or 5V).
- **Black:** Ground (GND).
- **Yellow / Green / Blue / Purple:** Signals and data lines (GPIO pins).

2.6 Safety Guidelines

Working with 3.3V microcontroller circuits is safe and will not deliver an electric shock, but observing a few basic rules will protect your components from damage:

- **Unplug before rewiring:** Always disconnect the USB cable from your computer before placing or moving components on the breadboard.
- **Avoid short circuits:** Never connect a jumper wire directly from the positive rail (3.3V or 5V) to the ground rail (GND). This causes a short circuit that can overheat the board.
- **Always use a series resistor with LEDs:** Directly connecting an LED between 3.3V and GND without a 220Ω resistor will burn out the LED.
- **3.3V logic levels:** The ESP32-C3 GPIO pins operate at 3.3V. Always power modules in this kit from the 3.3V pin unless a specific project instructs otherwise.
- **Disconnect if hot:** If any component feels hot to the touch or you notice a burning smell, immediately unplug the USB cable and ask an instructor to inspect your wiring.

Chapter 3: MicroPython IDE

3.1 Connecting Your Microcontroller

Throughout this guide, we will be making use of a free online IDE for connecting our microcontroller to our computer and loading, editing, and running code on it. You can access the IDE from your browser by going to <https://viper-ide.org/>. Click on the USB icon in the top right and choose your microcontroller from the list:

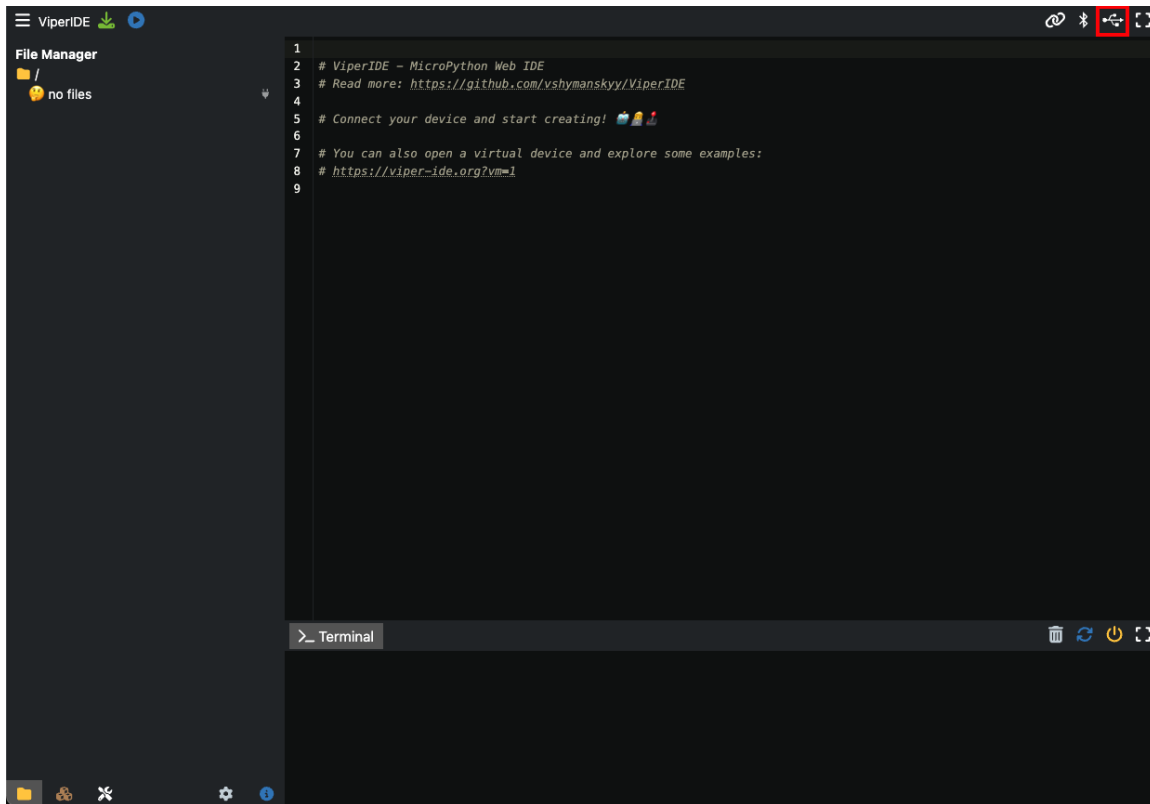


Figure 3.1: Click the button highlighted in red.

If you see multiple items in the dialog that pops up, choose the one that starts with "USB JTAG". See below for an example:

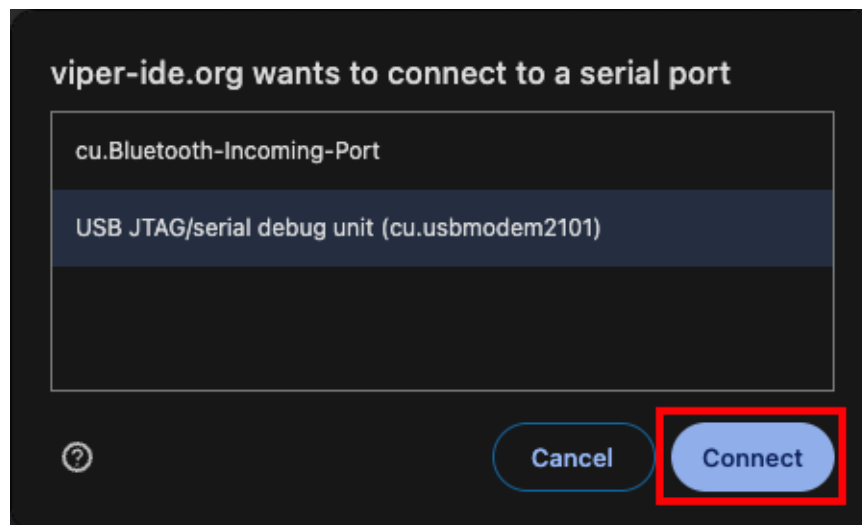


Figure 3.2: Click the button highlighted in red.

Once you have connected, you will see a green dialog labeled "Device connected" and the file manager on the list will populate with the list of files installed on the device:

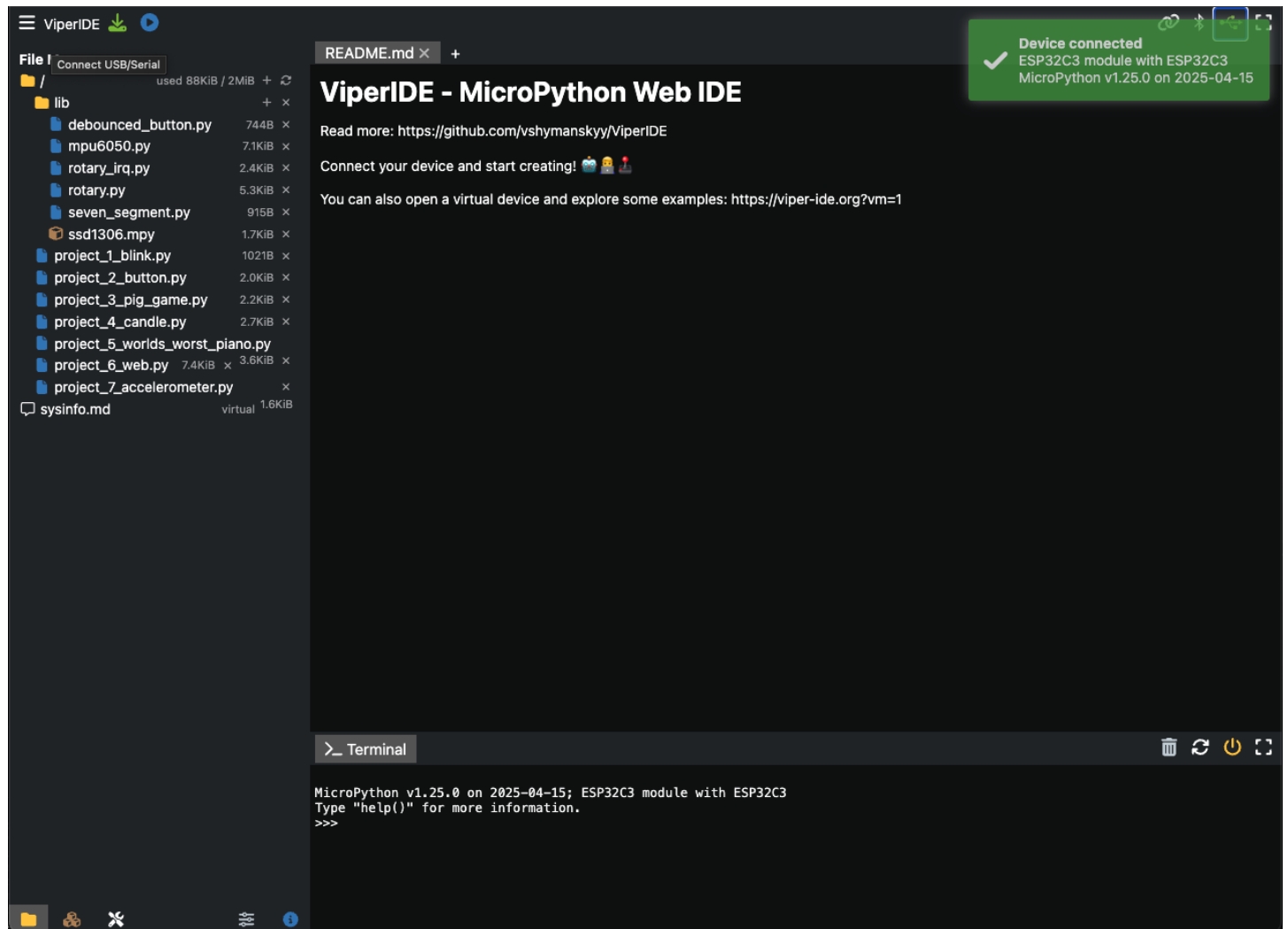


Figure 3.3: Make sure to open the right file for this project

After your device is connected, refer back to the chapter you are working on for the next steps.

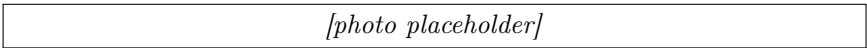
Chapter 4: Project 1: Night Light

4.1 Overview

This project is designed to provide a foundation for subsequent projects in this book (**and beyond**). Over the course of this project, you will:

- Create a simple circuit using your breadboard
- Write a program that runs in a loop
- Use MicroPython to read an analog sensor and drive an LED with PWM

At the end of this project, your microcontroller should run a MicroPython program that dims and brightens an LED as you cover and uncover a light sensor. Cover the sensor with your hand and the LED should get brighter—like a night light.



[photo placeholder]

Figure 4.1: The end result should look something like this

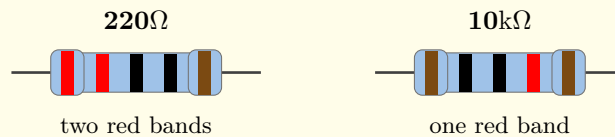
Learning goals:

- Build a first breadboard circuit with an LED, a current-limiting resistor, and an LDR
- Read a light level with the microcontroller's analog-to-digital converter (ADC)
- Use PWM (Pulse Width Modulation) to set LED brightness instead of only on or off

4.2 Components

- Seed XIAO ESP32-C3 microcontroller
- 1x LED (any color)
- 1x 220 Ω resistor (for the LED)
- 1x photoresistor (LDR)
- 1x 10k Ω resistor (for the LDR voltage divider)
- Breadboard
- Jumper wires

Which resistor is which? Both resistors are small and blue with five evenly spaced colored bands, and nothing is printed on them. You do not need to figure out which end to read from—just count the **red** bands. The **220 Ω** resistor has **two red bands next to each other**, and the **10k Ω** resistor has **exactly one red band**. Swapping the two will not damage anything, but the circuit will not behave the way this project describes, so ask an instructor if you are not sure.



4.3 Directions

4.3.1 Creating the circuit

Using jumper cables, you will assemble two small circuits: an LED with a current-limiting resistor, and a light sensor made from an LDR and a 10k Ω resistor.

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

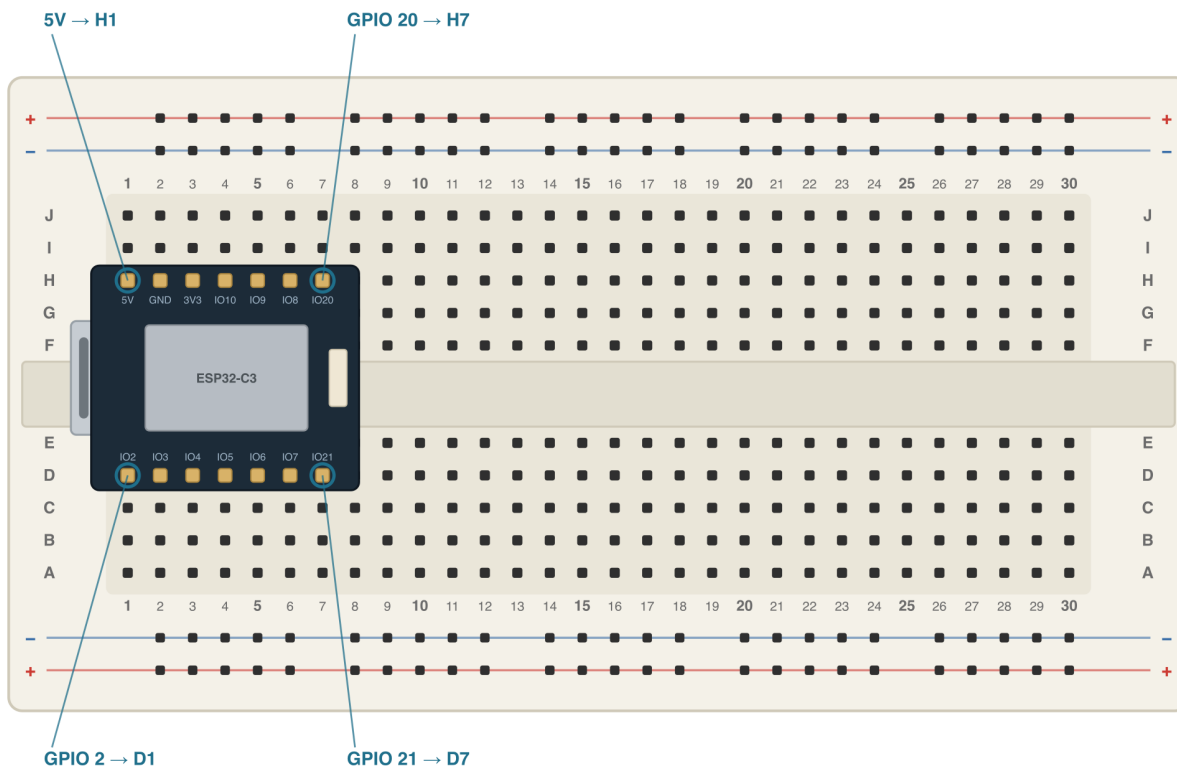


Figure 4.2: So far, so good!

Connect the LED and 220Ω resistor

Place an LED of your choice (the example image below uses RED) into the breadboard. The longer leg should be placed in **B16** and the shorter leg should be placed in **B17**. Then place one leg of a 220Ω resistor (doesn't matter which one) in **A17** and the other in **A21**.

You should be left with something that looks like this:

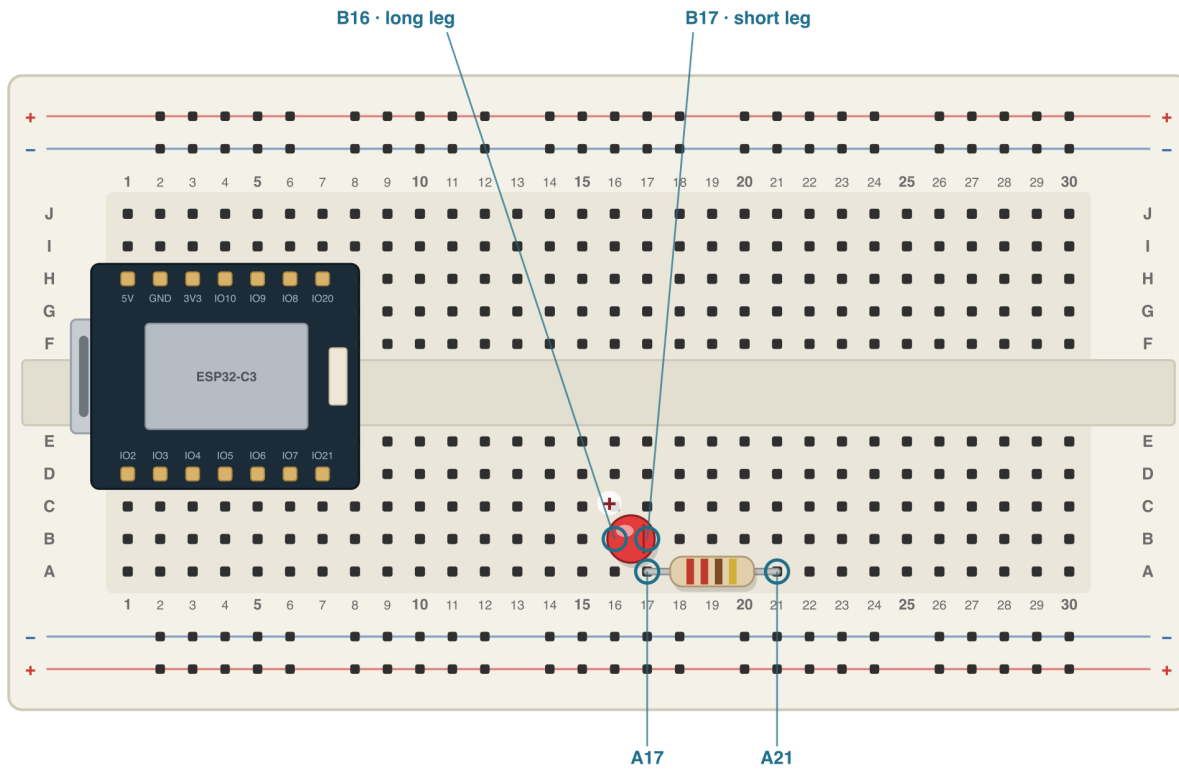


Figure 4.3: The LED and its current-limiting resistor are placed.

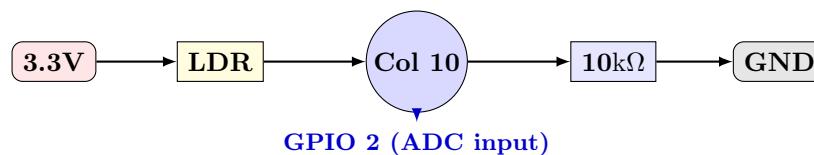
NOTE: Whenever you connect an LED to a circuit, you must always connect a resistor in series with it. In series means that the power goes in one leg of the LED, the other leg of the LED is attached to one leg of the resistor, and the ground completes the circuit from the other leg of the resistor. This is important so that you do not burn out the LED and/or the microcontroller pin.

Connect the LDR and 10kΩ resistor

An LDR (light-dependent resistor), also known as a photoresistor, is a sensor whose resistance changes with the light falling on it: brighter light lowers its resistance, while darkness raises it. Like standard resistors, an LDR has **no polarity**—either leg can face either way.

Place one leg of the LDR in **B10** and the other in **B11**. Then place one leg of the 10kΩ resistor in **A10** and the other in **A14**.

The LDR and the 10kΩ resistor share column 10 to form a **voltage divider**:



Because the microcontroller's analog pin measures **voltage** rather than resistance directly, this divider translates changing light into a changing voltage at column 10. When light shines on the LDR, its resistance decreases, pulling column 10 closer to 3.3V and increasing the number read by the ADC.

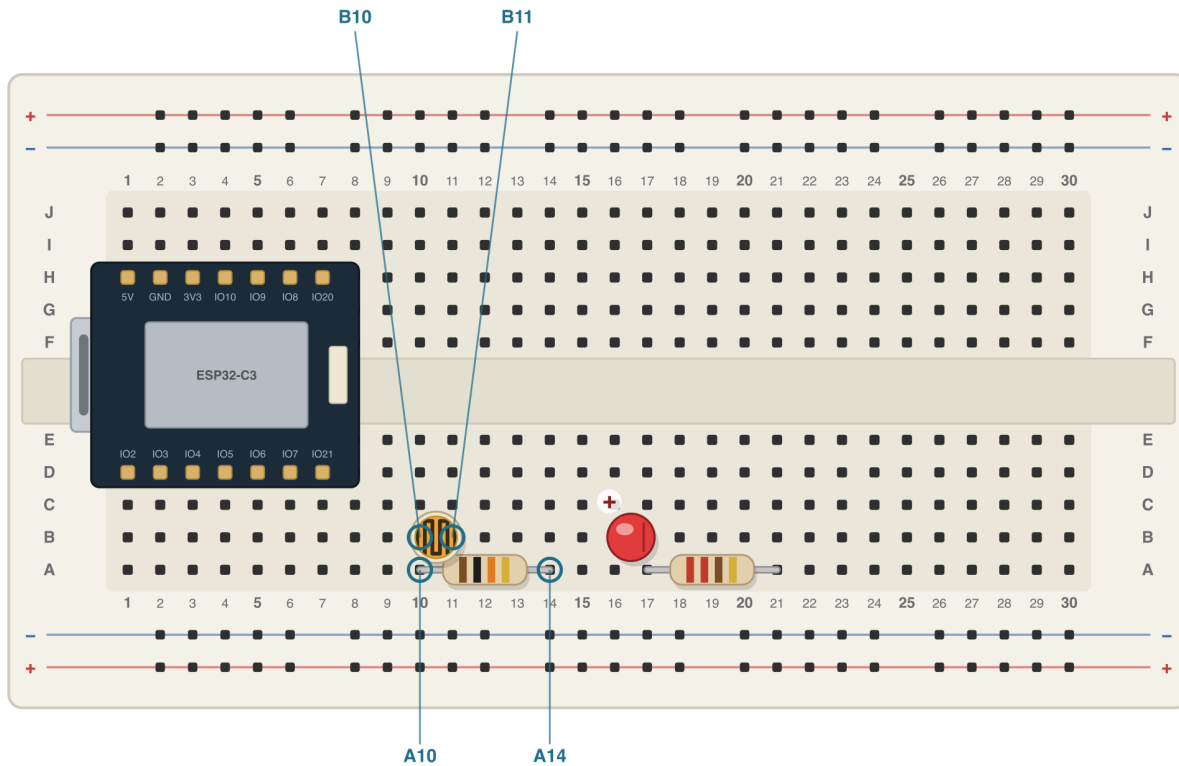


Figure 4.4: All of the components except for the jumper wires are now placed.

Connect the necessary jumper wires

Circuit connection	XIAO pin	Jumper holes
LED anode	GPIO 20	J7 → E16
LED resistor	GND	J2 → E21
LDR/resistor junction	GPIO 2	C1 → E10
LDR	3V3	J3 → E11
LDR resistor	GND	I2 → E14

The program uses PWM on GPIO 20 to set the LED brightness. GPIO 2 measures the voltage at the junction between the LDR and its 10kΩ resistor.

You should be left with something that looks like this:

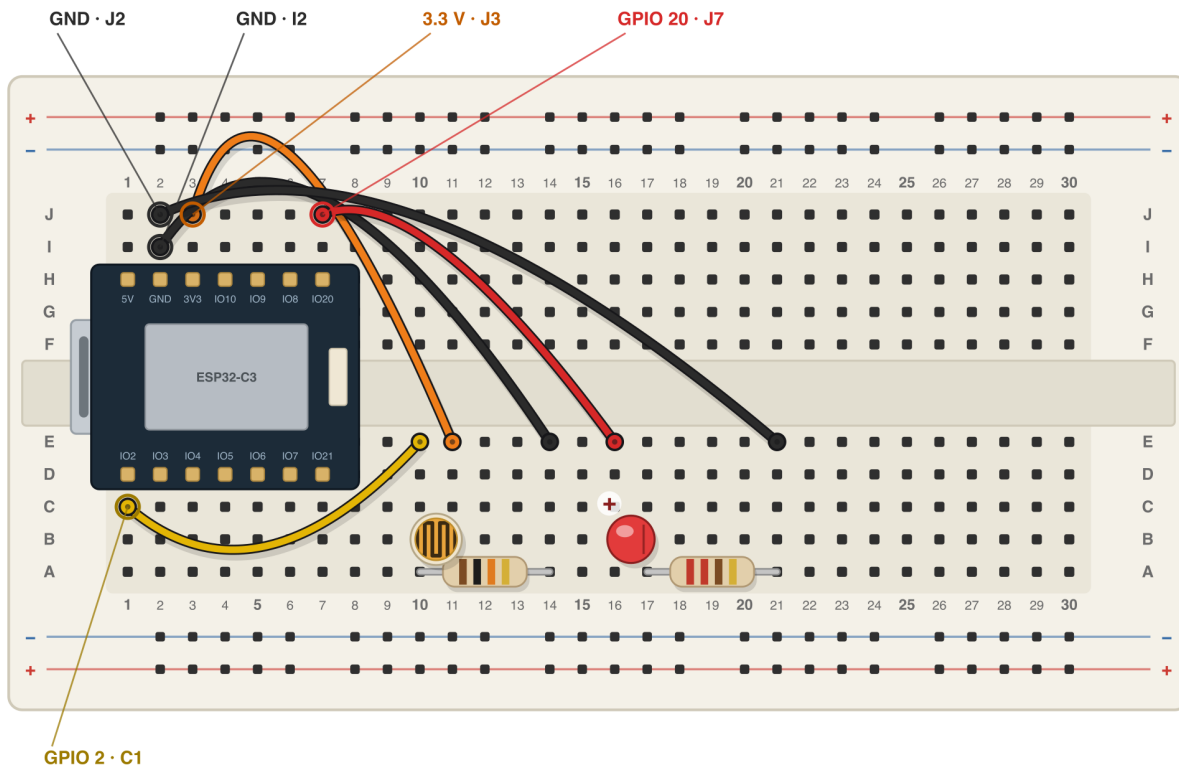


Figure 4.5: I’m absolutely POSITIVE I connected everything correctly!

With this wiring, more light on the LDR lowers its resistance, which **raises** the voltage (and the number) that GPIO 2 reads. The program inverts that number so a darker sensor makes a brighter LED.

4.3.2 Programming the microcontroller

Once all of the wiring is correct, connect the USB cable to the microcontroller and load the IDE to access it. Refer back to Chapter 3 for instructions.

Click on the file named “project_1_night_light.py”. This will load the code in the editor for this section. Read through the comments and the code to get a sense for how it works. Once you are ready, you can click the blue play button in the upper left of the window to start the script.

4.3.3 How the program works

The light sensor is read with the microcontroller’s ADC (analog-to-digital converter). That reading is a number from 0 to 4095.

The LED is driven with PWM (Pulse Width Modulation). Instead of leaving the pin fully on or fully off, PWM switches it on and off very quickly (here, 1000 times per second). The fraction of time it stays on is called the **duty cycle**. Your eye averages that into a brightness: duty 0 is off, duty 1023 is as bright as this pin can make the LED, and values in between are dimmer.

Each loop, the program reads the sensor, inverts the value so darkness becomes a larger duty, and updates the LED:

Listing 4.1: Mapping sensor reading to LED brightness

```
1 level = sensor.read()
2 duty = (ADC_MAX - level) * PWM_MAX // ADC_MAX
3 led.duty(duty)
4 print(level, duty)
```

The IDE console will print two numbers over and over: the raw sensor reading, then the PWM duty. Cover the LDR with your hand and you should see the first number go **down**, the second number go **up**, and the LED get brighter. Uncover it and the LED should dim.

You can stop the script by pressing the red stop button located where the blue play button used to be.

If something does not work:

- If the LED never lights, check LED polarity (longer leg toward GPIO 20 / hole **B16**), the 220Ω resistor, and the jumper from **J7** to **E16**.
- If the LED lights but does not change when you cover the sensor, check the LDR divider: GPIO 2 (**C1** to **E10**), 3.3V (**J3** to **E11**), and ground (**I2** to **E14**). Watch the printed numbers—if they never move, the sensor wiring is the problem.
- Classroom lighting varies. If the LED looks stuck bright or stuck dim, cover the LDR fully with your hand, then uncover it, and compare the two printed `level` values.

4.4 Review

In this project, we learned the basics of setting up a simple circuit and running some code to interact with that circuit. We learned that some pins can be programmed to turn connected devices on and off, and that PWM lets us control **how bright** an LED is, not only whether it is on. We also learned that an LDR plus a resistor makes a voltage divider the microcontroller can measure, and that connecting an LED in a circuit always requires adding a resistor in series to prevent burning out the components.

4.5 Possible Extensions

If you want to do some experimentation, try these:

- Swap the mapping so the LED is a **daytime indicator** (brighter when the room is brighter). Hint: use `duty = level * PWM_MAX // ADC_MAX` instead of subtracting from `ADC_MAX`.
- Add `LIGHT_MIN` and `LIGHT_MAX` constants from the numbers you printed, then scale the reading so covering and uncovering the sensor uses the full dim-to-bright PWM range.
- Slow the loop (`time.sleep`) or print less often if the serial console is too noisy.

Chapter 5: Project 2: Red Light, Green Light

5.1 Overview

In this project, you will build a reflex game with a button and a 3-pixel WS2812B LED strip. Each reaction window cycles through red, yellow, and green phases:

- During **RED**, do not press. Pressing during red ends the round immediately.
- During **YELLOW**, get ready for green and do not press yet.
- During **GREEN**, press the button as quickly as possible to score a point.

Each successful green press prints your reaction time in milliseconds to the serial console. Each round has 5 total reaction windows. A hit during green scores a point, and a missed green window counts as a miss.

Learning goals:

- Read a pushbutton using an internal pull-up resistor
- Control WS2812B addressable LEDs with MicroPython
- Build a simple game state machine
- Measure reaction time with millisecond timers

5.2 Components

- Seed XIAO ESP32-C3 microcontroller
- 3-pixel WS2812B LED strip
- 1x pushbutton
- Breadboard
- Jumper wires

5.3 Directions

Remove previous components

Before starting Project 2, remove components and wires from the previous project.

5.3.1 Creating the circuit

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

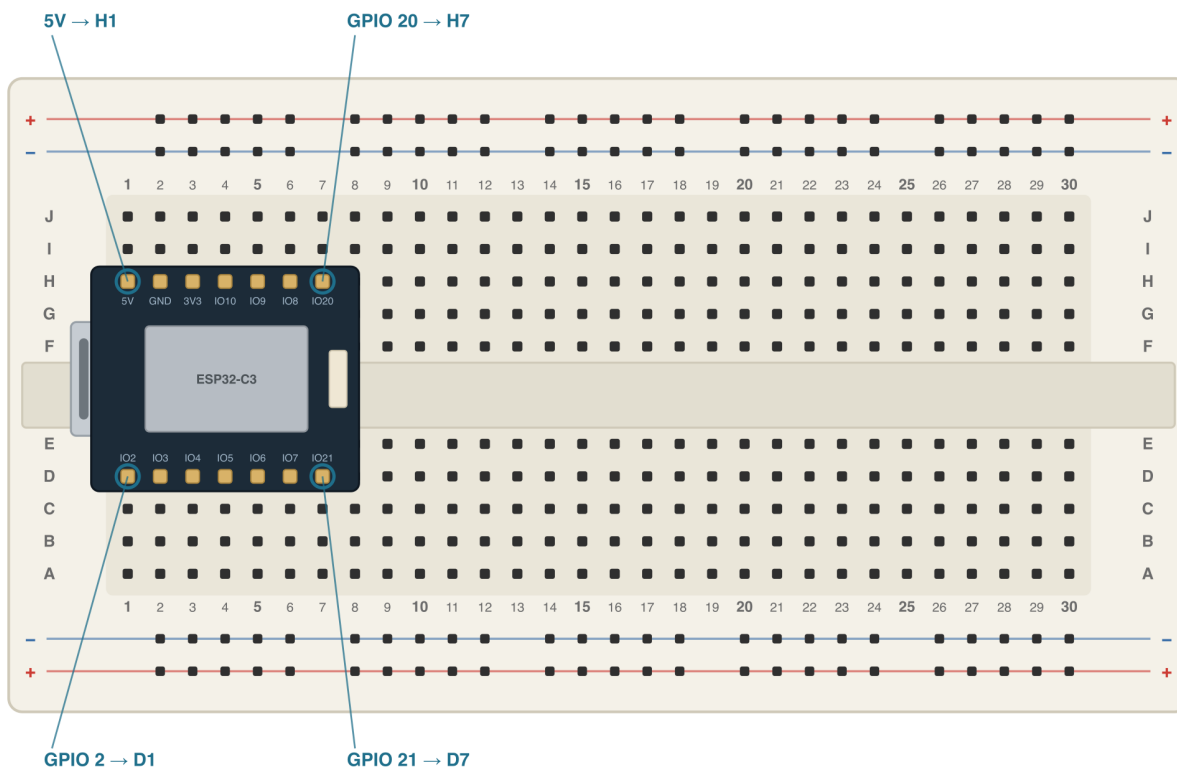


Figure 5.1: So far, so good!

WS2812B strip wiring

The LED strip has a right-angle header so it stands upright once seated. Push those three pins into the top half of the board: **5V** into **J12**, **DI** into **J13**, and **GND** into **J14**. Then add these connections:

WS2812B pin	XIAO pin	Jumper holes
5V	3V3	I3 → I12
DI	GPIO 20	I7 → I13
GND	GND	I2 → I14

Button wiring

Press the button into the bottom edge of the board so that its legs reach from column **A** up to column **D**. One pair of legs goes into **A12** and **D12**, and the other pair two rows along into **A14** and **D14**. The two legs of a pair are already joined inside the button, so it is the gap between the two pairs that the button closes. Then add these connections:

Button connection	XIAO pin	Jumper holes
One side	GPIO 21	C7 → E12
Other side	GND	J2 → E14

The code enables the internal pull-up resistor, so the pin reads HIGH when idle and LOW when pressed.

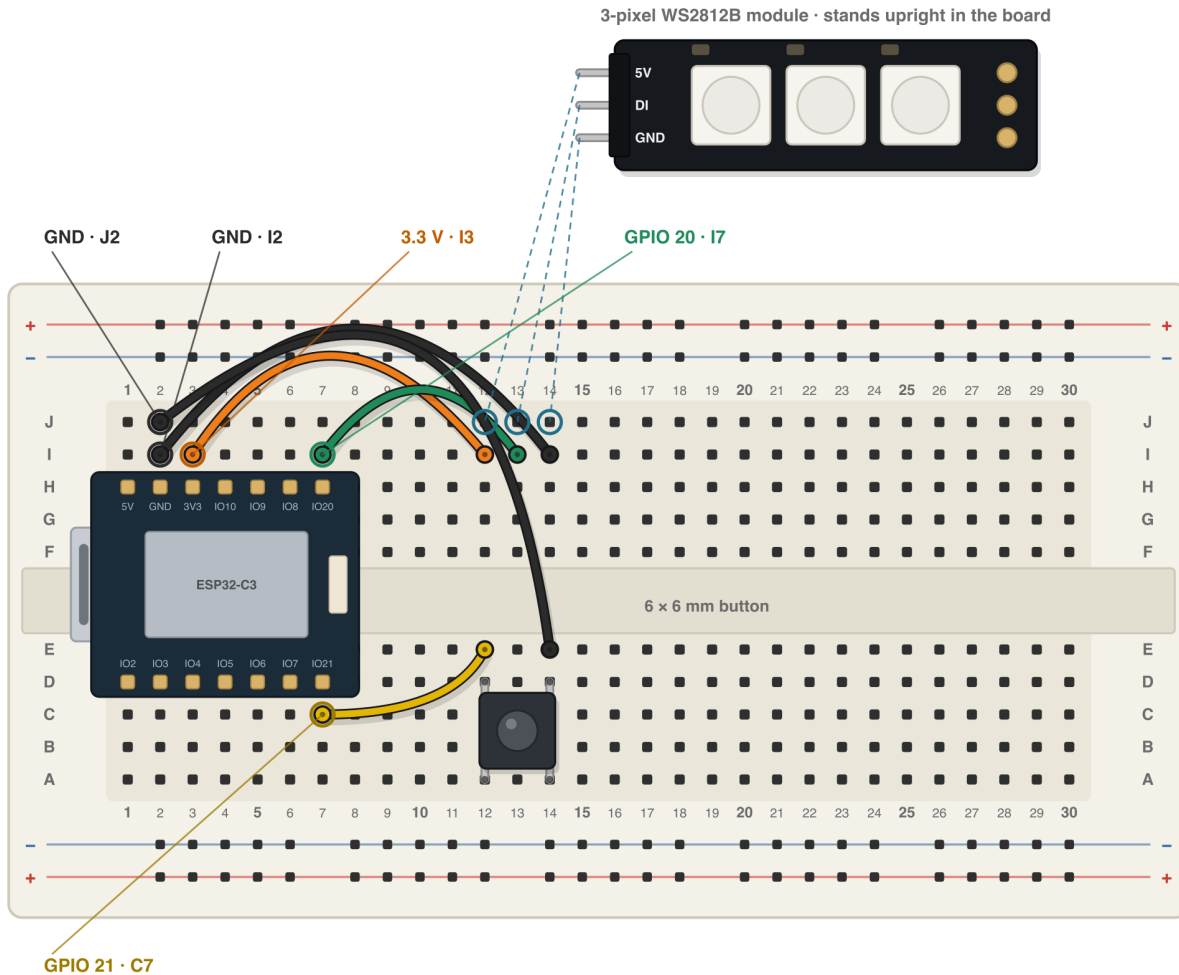


Figure 5.2: Connect the LED strip and button to the XIAO ESP32-C3.

5.3.2 Programming the microcontroller

Open the file `project_2_red_light_green_light.py` and run it in the IDE (Chapter 3).

At startup, the serial console will print instructions and wait for a button press to begin:

Project 2: Red Light, Green Light
Press button to start.

During gameplay:

- The 3 LEDs act like a stoplight: red LED, yellow LED, and green LED are used individually.
- RED phase lights only the red LED and prints “RED! Do not press.”
- YELLOW phase lights only the yellow LED and prints “YELLOW! Get ready...”
- GREEN phase lights only the green LED and prints “GREEN! Press now.”
- Idle/round-over uses the yellow LED as a waiting indicator.
- A valid green press prints reaction time and increases score.
- A missed green window is counted as one of the 5 windows.
- A red/yellow press ends the round and prints final score + best reaction.

5.3.3 Hardware test checklist

Use this checklist to confirm your circuit and code are working:

1. Confirm LED order on your strip: the LED closest to the header is red, the middle is yellow, and the far end is green.
2. Press button from idle state: yellow wait indicator transitions into red/green gameplay cycling.
3. Wait for GREEN (green LED only) and press quickly: score increases and a reaction time is printed.
4. Do not press during RED or YELLOW: pressing before green ends the round immediately.
5. Skip pressing during one green window: verify it is counted as a miss and the next window starts.
6. Repeat at least 5 rounds to confirm stable behavior and reliable button detection.

5.4 Review

In this project, you combined timed logic, button input, and LED output to build a full game loop. You practiced:

- State-based programming (idle, green, red, round_over)
- Reusing a shared debounced button library for cleaner input handling
- Using `time.ticks_ms()` and `time.ticks_diff()` for timing
- Providing immediate visual and text feedback to players

5.5 Possible Extensions

Try these enhancements:

- Add difficulty levels by shrinking GREEN timing windows
- Display average reaction time after each round
- Add a buzzer sound effect for valid and invalid presses
- Use different LED patterns (chase, blink, gradient) per game state
- Add a countdown before each round starts

Chapter 6: Project 3: Safe Cracker

6.1 Overview

In this project, you will build a combination-lock game using a rotary encoder and a small speaker. Spin the encoder dial to select digits 0–9, press the built-in encoder button to lock in each digit, and listen for audio hints that get warmer as you approach the correct value. Match the full secret sequence to “open the safe!”

Over the course of this project, you will:

- Wire a KY-040 rotary encoder module to your microcontroller
- Wire a small speaker using PWM (Pulse Width Modulation) to generate tones
- Use library code to reliably read encoder rotation and button presses
- Program a game loop with randomized secret codes and audio proximity feedback
- Practice reading serial output to debug your program

At the end of this project, your microcontroller will run a fully playable combination-lock game entirely driven by sound and a single knob—no screen required!

How the game works: A random 3-digit secret code (each digit 0–9) is generated when the program starts and printed to the serial console. Spin the encoder knob to change the current dial value. As you get closer to the target digit, the tone played on each tick gets higher and shorter. When you are exactly on the target, you hear a sharp high click. Press the encoder button to lock in your digit. A correct digit advances you to the next position; a wrong digit plays a low buzz and you try again. Crack all three digits to trigger the victory fanfare and start a new game!

6.2 Components

- Seed XIAO ESP32-C3 microcontroller
- KY-040 rotary encoder module (5-pin, includes onboard pull-up resistors)
- Treedix 1W 8 Ω mini speaker (JST 2-pin connector)
- Breadboard
- Jumper wires

6.3 Directions

Remove previous components

Before beginning, remove any components from prior chapters including LEDs, buttons, and wires. You may leave the microcontroller attached to the breadboard.

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

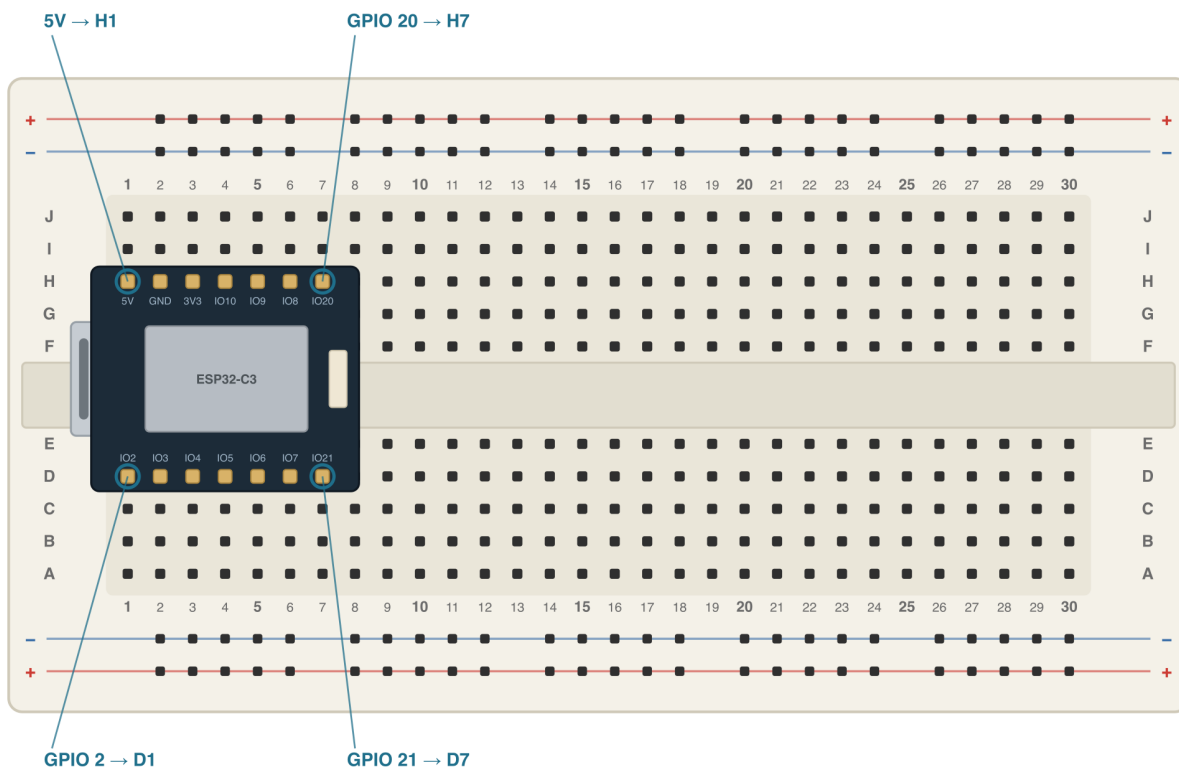


Figure 6.1: So far, so good!

6.3.1 Understanding the KY-040 Rotary Encoder

A rotary encoder is a type of position sensor that converts the angular position (rotation) of a knob into electrical signals. Unlike a potentiometer, an encoder has no physical stops—you can spin it continuously in either direction.

The KY-040 module has five pins:

- **CLK** — Clock signal; pulses as the knob turns
- **DT** — Data signal; used together with CLK to determine direction
- **SW** — Switch; goes LOW when the built-in button is pressed
- **+** — Power supply (connect to 3.3V); some boards label this **VCC**
- **GND** — Ground

Hold the module with the knob facing you and the pins pointing down: the silkscreen reads **GND**, **+**, **SW**, **DT**, **CLK** from left to right.

Important: Power the KY-040 from **3.3V**, not 5V. The ESP32-C3 GPIO pins are not 5V-tolerant, so using 5V could damage your microcontroller.

6.3.2 Creating the circuit

Using jumper cables, assemble the circuit according to the following wiring table. The encoder and speaker share the breadboard with the microcontroller.

KY-040 Encoder Wiring

Seat the encoder module's five-pin header along the front edge of the breadboard, knob facing you, so the pins land in silkscreen order: **GND** in **A12**, **+** in **A13**, **SW** in **A14**, **DT** in **A15**, and **CLK** in **A16**. Row A leaves the knob clear of the other wiring, so it is easier to grip and turn. Then add these connections:

KY-040 pin	XIAO pin	Jumper holes
GND	GND	I2 → E12
+ (VCC)	3V3	I3 → E13
SW	GPIO 8	I6 → E14
DT	GPIO 9	I5 → E15
CLK	GPIO 10	I4 → E16

Speaker Wiring

Insert the speaker's adapted Dupont leads into the bottom half of the breadboard: the positive lead in **C20** and the negative lead in **C22**. Then add these connections:

Speaker lead	XIAO pin	Jumper holes
Signal (+)	GPIO 20	I7 → E20
Ground (−)	GND	J2 → E22

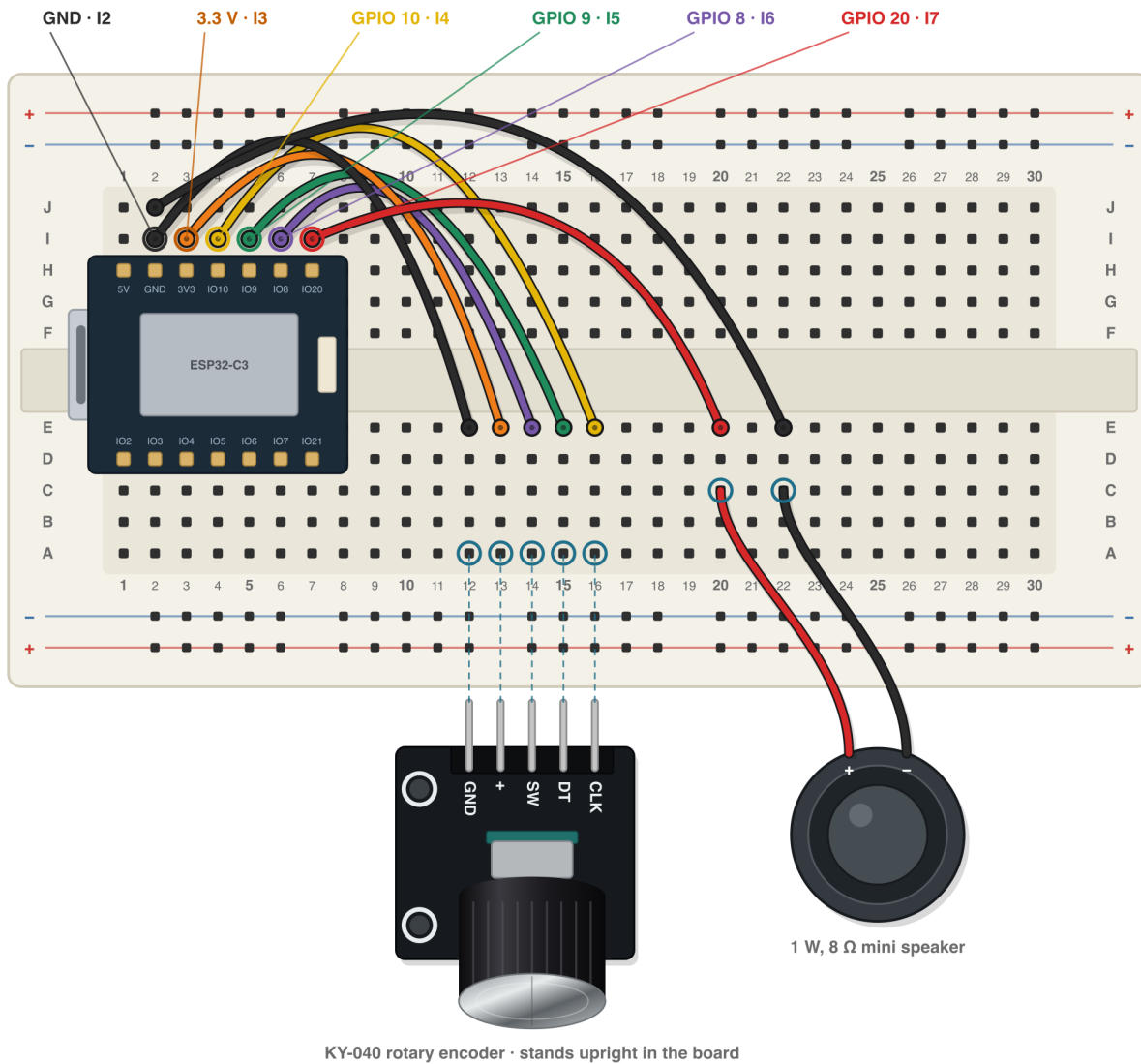


Figure 6.2: Encoder and speaker wired to the XIAO ESP32-C3

6.3.3 Understanding the Code

Open the file named `project_3_safe_cracker.py` in the IDE. Let's walk through the key sections.

Configuration Constants

At the top of the file, a configuration block defines all pin assignments and game parameters in one place. This makes it easy to adjust the game without hunting through the code:

Listing 6.1: Configuration constants

```

1 CODE_LENGTH = 3
2 DIAL_MIN = 0
3 DIAL_MAX = 9
4
5 PIN_CLK = 10
6 PIN_DT = 9
7 PIN_SW = 8
8 PIN_SPEAKER = 20
9
10 TONE_EXACT = (880, 80) # high click when exactly on target
11 TONE_WARM = (550, 50) # warm mid tone when 1-2 away
12 TONE_COLD = (220, 30) # low cold tone when 3+ away

```

```

13 TONE_BUZZ    = (150, 200) # wrong-digit buzz
14 VICTORY_NOTES = [(262, 150), (330, 150), (392, 150), (523, 400)]

```

Playing Tones with PWM

The `play_tone` function uses PWM (Pulse Width Modulation) to drive the speaker. PWM rapidly switches the GPIO pin on and off at a given frequency, which the speaker cone turns into sound:

Listing 6.2: PWM tone helper

```

1 def play_tone(freq, duration_ms):
2     pwm = PWM(Pin(PIN_SPEAKER), freq=freq, duty=512)
3     time.sleep_ms(duration_ms)
4     pwm.deinit()

```

A duty of 512 out of 1023 means the pin is HIGH roughly half the time—this is called a 50% duty cycle and produces the loudest sound for a square wave.

Proximity Hints

The game measures how close the current dial value is to the target digit using a *circular distance*. Because the dial wraps around (0 and 9 are neighbors), we take the smaller of the straight-line distance and the wrap-around distance:

Listing 6.3: Circular distance on a 0-9 dial

```

1 def dial_distance(current, target):
2     diff = abs(current - target)
3     return min(diff, 10 - diff)

```

This distance is then mapped to a tone:

Distance to target	Sound
0 (exact match)	High click (880 Hz, 80 ms)
1 or 2 away	Warm mid tone (550 Hz, 50 ms)
3 or more away	Low cold tone (220 Hz, 30 ms)

Reading the Encoder and Button

The code uses two library modules from the `lib/` folder:

- **RotaryIRQ** — reads the encoder using hardware interrupts; configured for bounded range 0–9 with automatic wrap-prevention.
- **DebouncedButton** — reads the encoder’s push-button and fires a callback only once per press, filtering out electrical noise (bouncing).

Listing 6.4: Setting up the encoder and button

```

1 knob = RotaryIRQ(
2     pin_num_clk=PIN_CLK,
3     pin_num_dt=PIN_DT,
4     min_val=DIAL_MIN,
5     max_val=DIAL_MAX,
6     reverse=True,
7     pull_up=True,
8     range_mode=RotaryIRQ.RANGE_BOUNDED,
9 )
10 knob.add_listener(on_dial_change)
11 DebouncedButton(PIN_SW, on_enter)

```

`reverse=True` makes clockwise rotation increase the displayed number (adjust to `False` if your knob spins the wrong way). `pull_up=True` enables the internal pull-up resistors so the CLK and DT pins read HIGH when idle.

Game Loop

Each time the knob moves, `on_dial_change` is called automatically by the library. It reads the current encoder value, computes the distance to the target digit, and plays the corresponding hint tone. When the encoder button is pressed, `on_enter` checks whether the current value matches the target:

- **Correct:** A confirming click plays, the dial index advances. If all digits are correct, the victory fanfare plays and a new game begins.
- **Wrong:** A low buzz plays and the player tries again on the same digit.

6.3.4 Programming the microcontroller

Connect the USB cable to the microcontroller and open the IDE. Refer back to Chapter 3 for IDE instructions if needed. Click on the file named `project_3_safe_cracker.py`. Read through the comments and code to understand how it works, then click the blue play button to start the game.

When the program starts, watch the serial console—it will print the secret code so you (or your instructor) can verify the game is working:

New safe code: [3, 7, 1]

Dial 1/3: showing 5 (target 3, distance 2)

Tip: If the tones are silent, check that the speaker’s signal wire is connected to GPIO 20 and that the ground wire goes to GND. If the dial counts in the wrong direction, change `reverse=True` to `reverse=False` in the code.

6.4 Review

In this project, we combined hardware input (rotary encoder) and output (PWM speaker tones) to build a complete interactive game. We learned:

- How a rotary encoder converts physical rotation into digital signals
- How PWM can drive a speaker to play different pitches
- How to use interrupt-driven library code to read hardware without blocking the main loop
- How circular distance can create smooth “hot and cold” proximity feedback
- How to structure a game with state variables, callbacks, and a main loop

6.5 Possible Extensions

If you want to experiment further, try these ideas:

- Increase `CODE_LENGTH` to 4 or 5 digits for a harder challenge
- Add a countdown timer—if you don’t crack the safe in time, the game resets
- Change the victory fanfare to play a song you recognize (look up the note frequencies!)
- Add a “lives” system: after 3 wrong guesses on the same digit, the game resets
- Print the elapsed time to the serial console when the safe is cracked

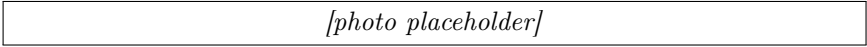
Chapter 7: Project 4: Pocket DJ

7.1 Overview

In this project, you will turn your microcontroller into a handheld musical instrument: the **Pocket DJ!** You will wire three pushbuttons, a mini speaker, and a 3-pixel RGB LED strip to your breadboard. Pressing each button triggers a distinct musical note on the speaker and lights up a matching colored pixel on the LED strip.

Over the course of this project, you will:

- Connect multiple tactile pushbuttons and read them using MicroPython
- Use PWM (Pulse Width Modulation) to generate different pitch frequencies on a speaker
- Control individual addressable WS2812B RGB LEDs to create synchronized visual pulses
- Program rhythm loops and sound sequences using Python lists and tuples



[photo placeholder]

Figure 7.1: The assembled Pocket DJ ready to drop some beats!

Learning goals:

- Read pushbuttons with active-low logic and internal pull-up resistors
- Generate musical notes by controlling PWM frequency on a speaker
- Control WS2812B addressable LEDs using the `neopixel` library
- Structure musical sequences and automated rhythm loops in code

7.2 Components

- Seeed XIAO ESP32-C3 microcontroller
- 3-pixel WS2812B LED strip
- 3x Tactile pushbuttons (6 × 6 mm)
- 1x Treedix 1W 8Ω mini speaker (with breadboard-friendly Dupont leads)
- Breadboard
- Jumper wires

7.3 Directions

Remove previous components

Before beginning, remove components and wires from the previous project. You may leave the microcontroller seated in the breadboard.

7.3.1 Creating the circuit

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

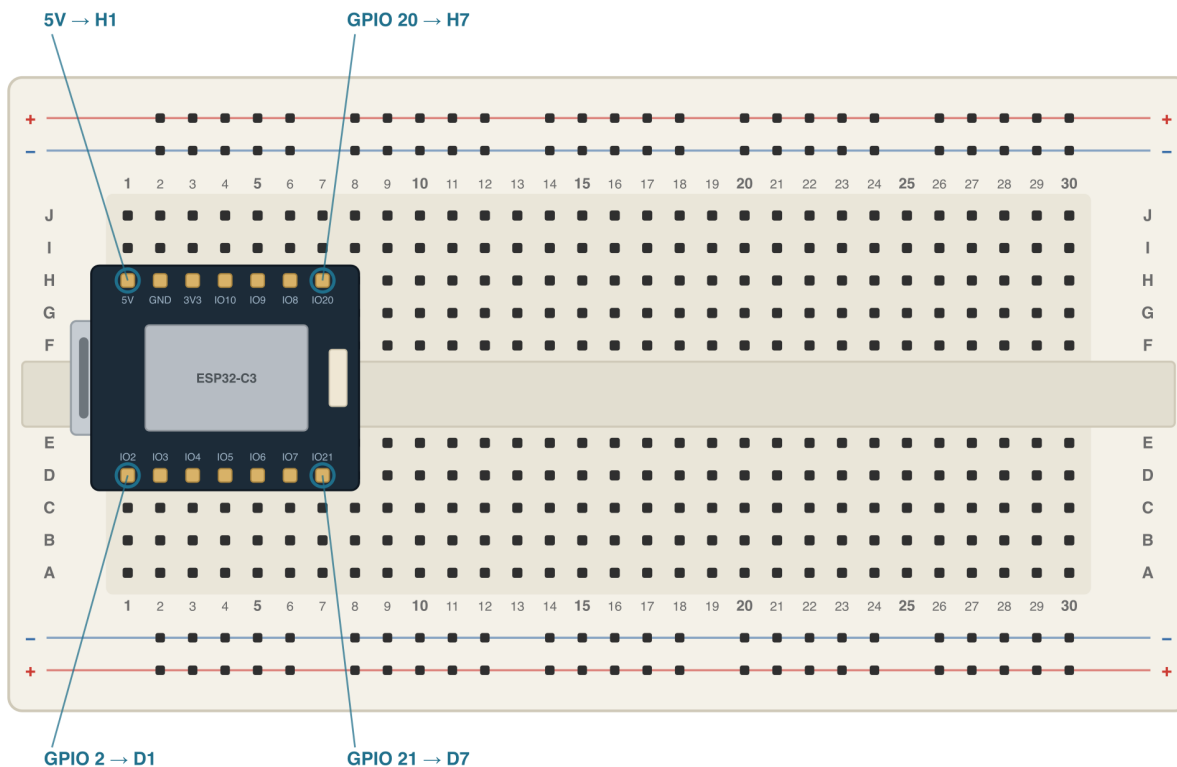


Figure 7.2: So far, so good!

Ground rail distribution

Because we have multiple components requiring a ground connection, we connect the microcontroller's **GND** pin to the breadboard's top blue (–) power rail. This powers the entire rail with ground:

Connection	XIAO pin	Jumper holes
Top GND rail	GND	I2 → top-2

WS2812B LED strip placement and wiring

The LED strip has a right-angle header so it stands upright once seated. Insert those three pins into row **F** (rows 22–24) on the right side of the breadboard so it is clearly visible and not obscured by jumper wires: **5V** into **F22**, **DI** into **F23**, and **GND** into **F24**. Then connect the jumper wires:

WS2812B pin	Source / Pin	Jumper holes
5V	3V3	I3 → J22
DI	GPIO 20	I7 → J23
WS2812B ground	Top GND rail	top-24 → J24

Tactile button placement and wiring

Seat the three pushbuttons in the lower half of the breadboard spanning columns **A** through **D**:

- **Pad 0 (Loop/Bass):** Place one pair of legs in **A10** and **D10**, and the other pair in **A12** and **D12**.
- **Pad 1 (Mid note):** Place one pair of legs in **A14** and **D14**, and the other pair in **A16** and **D16**.
- **Pad 2 (High note):** Place one pair of legs in **A18** and **D18**, and the other pair in **A20** and **D20**.

Connect each button's signal leg (column **E**) to the microcontroller:

Button signal	XIAO pin	Jumper holes
Pad 0 (row 10)	GPIO 21	C7 → E10
Pad 1 (row 14)	GPIO 7	C6 → E14
Pad 2 (row 18)	GPIO 6	C5 → E18

Next, wire each button's ground leg (column **E**) to the top ground rail:

Button ground	Source	Jumper holes
Pad 0 ground	Top GND rail	top-12 → E12
Pad 1 ground	Top GND rail	top-16 → E16
Pad 2 ground	Top GND rail	top-20 → E20

Speaker placement and wiring

Insert the speaker's two Dupont leads into the lower half of the breadboard: the positive red lead into **C26** and the negative black lead into **C28**. Then connect the speaker signal and ground jumpers:

Speaker lead	Source / Pin	Jumper holes
Signal (+)	GPIO 10	I4 → E26
Speaker ground (−)	Top GND rail	top-28 → E28

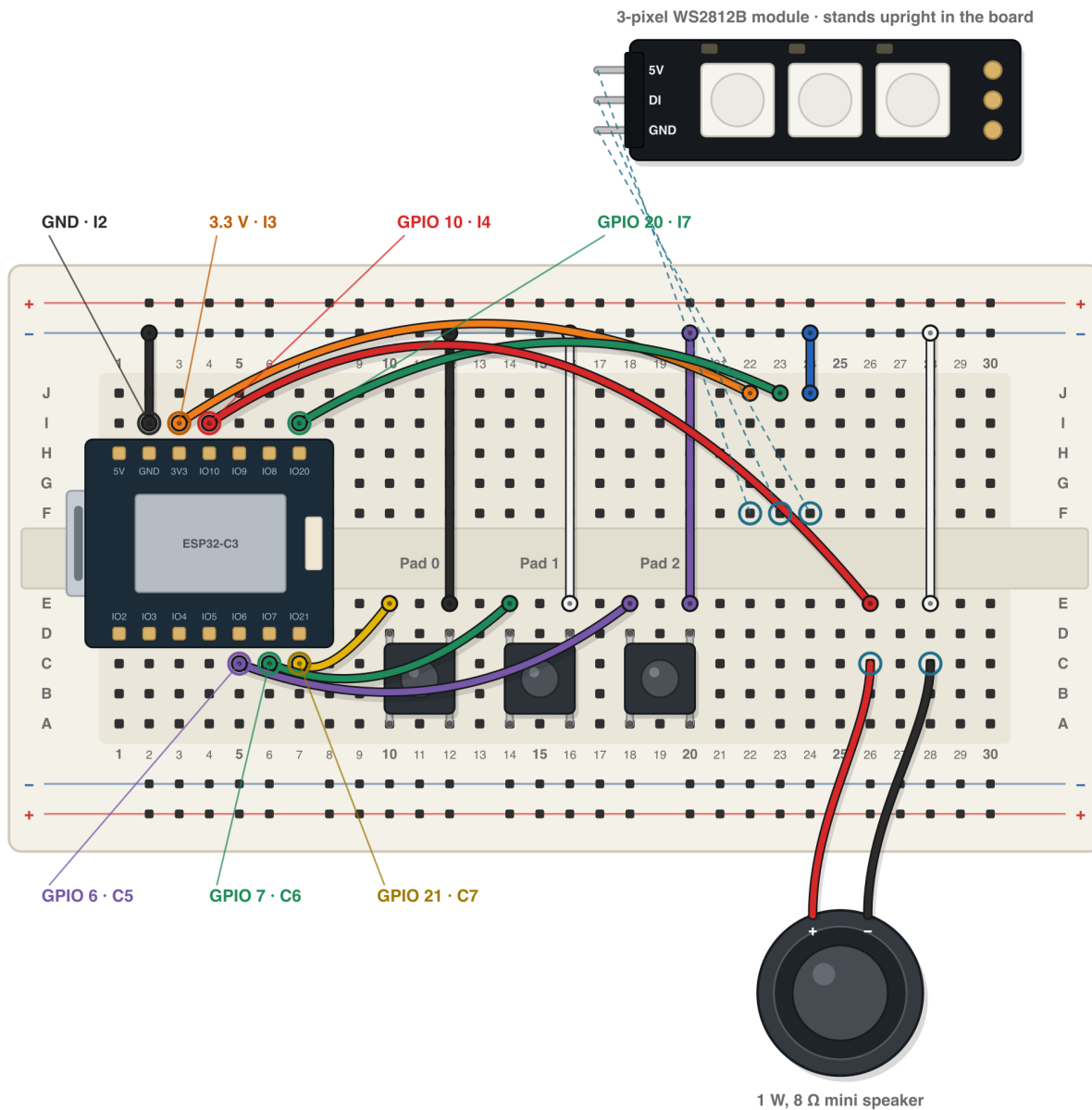


Figure 7.3: Complete wiring diagram for Project 4: Pocket DJ.

7.3.2 Programming the microcontroller

Open the file named `project_4_pocket_dj.py` in the IDE and run it.

When the script starts, the serial console displays:

Project 4: Pocket DJ

Pad 0: GPIO 21 | Pad 1: GPIO 7 | Pad 2: GPIO 6

Speaker: GPIO 10 | LED Strip: GPIO 20

Press any button to play music!

Try pressing each pad:

- **Pad 1** plays a middle note (330 Hz) and flashes the middle LED green.
- **Pad 2** plays a higher note (392 Hz) and flashes the third LED blue.
- **Pad 0** plays an automated 4-step rhythm loop across the notes and colors!

7.3.3 How the program works

Musical tones with PWM

In Project 1, PWM was used to control LED brightness by changing duty cycle at a fixed frequency. Here, we use PWM on the speaker pin, but instead of changing brightness, we change the **frequency** (the pitch):

Listing 7.1: Playing a tone on the speaker

```
1 pwm = PWM(Pin(PIN_SPEAKER), freq=freq, duty=512)
2 time.sleep_ms(duration_ms)
3 pwm.deinit()
```

A higher frequency makes the speaker vibrate faster, producing a higher musical pitch. A 50% duty cycle (`duty=512`) provides optimal volume. Calling `pwm.deinit()` silences the speaker when the note is done.

RGB LEDs with NeoPixel

The WS2812B LEDs are addressable: a single digital data pin (GPIO 20) controls the color and brightness of all three pixels independently. Colors are specified as (**Red**, **Green**, **Blue**) tuples from 0 to 255:

Listing 7.2: Lighting a pixel

```
1 pixels[pixel_idx] = color
2 pixels.write()
```

Rhythm loops

Pad 0 reads a sequence of note indices and timing delays stored in a Python tuple:

Listing 7.3: Rhythm loop sequence

```
1 LOOP = (
2     (0, 80), # Pad 0, rest 80 ms
3     (2, 60), # Pad 2, rest 60 ms
4     (1, 80), # Pad 1, rest 80 ms
5     (2, 120), # Pad 2, rest 120 ms
6 )
```

When triggered, a `for` loop steps through each element, playing each note and waiting before the next beat.

7.3.4 Hardware test checklist

1. Press Pad 1: You should hear a tone and see the green LED light up.
2. Press Pad 2: You should hear a higher tone and see the blue LED light up.
3. Press Pad 0: You should hear the 4-step beat loop play across the different tones and LEDs.
4. Check serial output in the IDE to verify that button presses are detected cleanly without double triggers.

If something does not work:

- **No sound from speaker:** Check that the speaker red lead is in **C26** and jumpered from **I4** to **E26** (GPIO 10). Check that the black lead is in **C28** and grounded to the top rail via **top-28** to **E28**.
- **LED strip does not light:** Check that the strip is seated in F22–F24 with DI in **F23** and connected to GPIO 20 (**I7** to **J23**), power to 3.3V (**I3** to **J22**), and GND to the top rail (**top-24** to **J24**).
- **Button press does nothing:** Verify button orientation and connections. Check that the microcontroller GND wire (**I2** to **top-2**) is connected to the top rail, and each button ground jumper is connected.

7.4 Review

In this project, you constructed a multi-component digital instrument combining inputs (pushbuttons) with two different types of outputs (PWM audio on a speaker and addressable RGB LEDs). You learned how frequency determines pitch, how to drive NeoPixel LEDs, and how to program sequenced loops.

7.5 Possible Extensions

- **Custom melodies:** Modify the `LOOP` tuple in `project_4_pocket_dj.py` to create your own custom beats or jingles.
- **Scale selector:** Add higher or lower octaves by doubling or halving the frequencies in `PADS`.
- **DJ light show:** Update `play_note` to create visual ripple effects where non-playing LEDs pulse with complementary colors.

Chapter 8: Project 5: Wi-Fi Mood Lamp

8.1 Overview

In this project, your ESP32-C3 becomes both a Wi-Fi access point and a light controller. After uploading a Python file plus one HTML dashboard file, you will connect your phone or laptop to the microcontroller's network, open a local web page, and control a chain of three WS2812B LEDs.

You will build:

- A password-protected Wi-Fi network hosted by the ESP32-C3
- A local web dashboard served directly from the board
- Manual lighting controls for all LEDs together and for each LED individually
- A set of built-in animations (breathing, rainbow, wipe, and blink)

By the end, you will have a complete wireless “mood lamp” that works without internet.

8.2 Components

- Seed XIAO ESP32-C3 microcontroller
- 3-pixel WS2812B LED strip
- Breadboard
- Jumper wires

8.3 Directions

8.3.1 Creating the circuit

Before starting, remove components from earlier projects that are no longer needed.

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

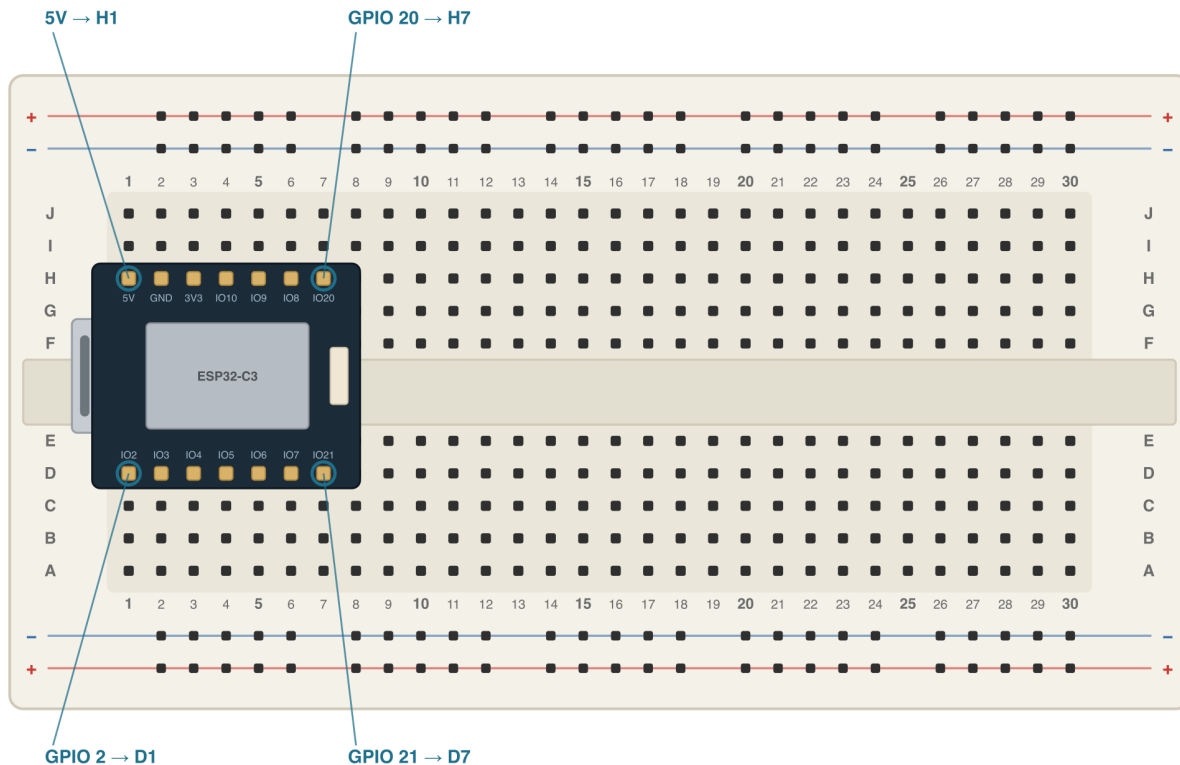


Figure 8.1: So far, so good!

WS2812B strip wiring

The LED strip has a right-angle header so it stands upright once seated. Push those three pins into the top half of the board: **5V** into **J12**, **DI** into **J13**, and **GND** into **J14**. Then add these connections:

WS2812B pin	XIAO pin	Jumper holes
5V	5V	I1 → I12
DI	GPIO 20	I7 → I13
GND	GND	I2 → I14

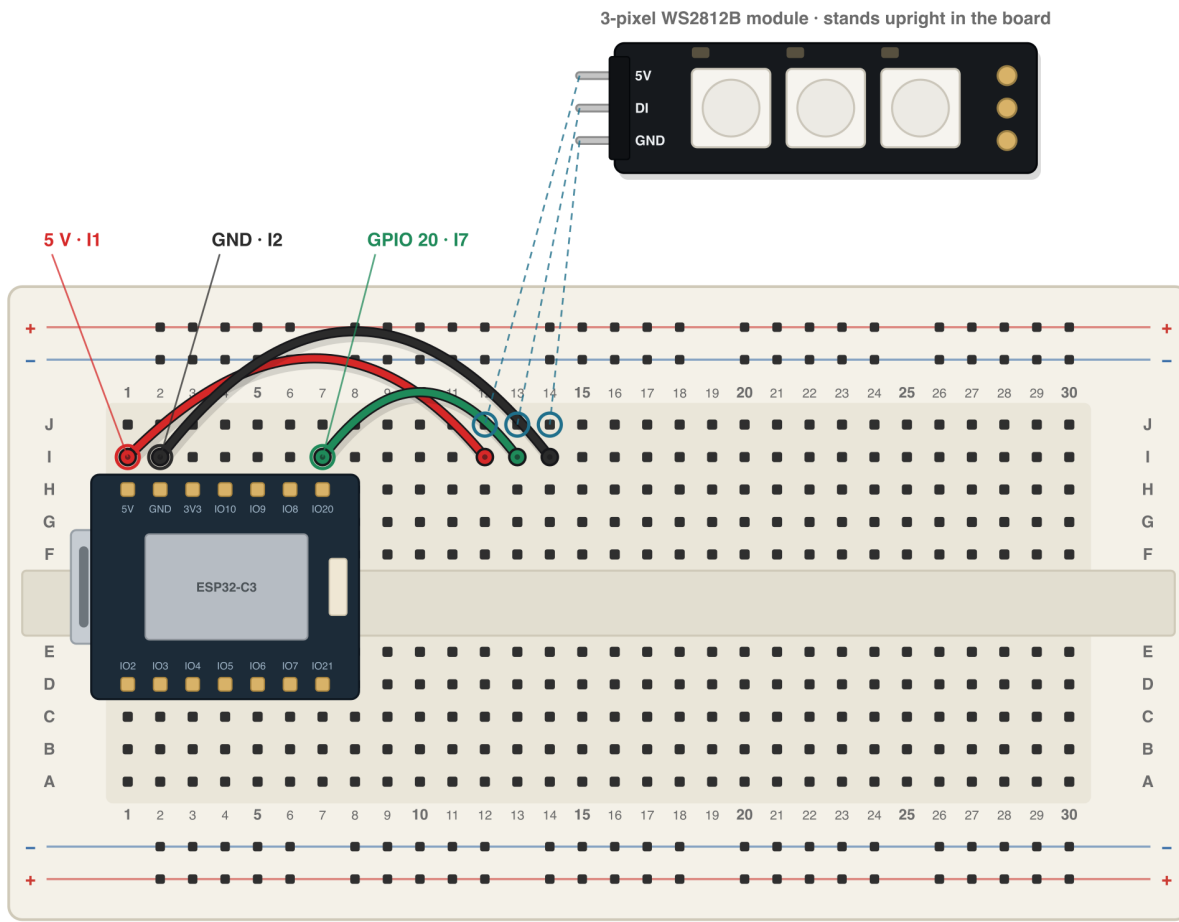


Figure 8.2: Connect the three-pixel LED strip to the XIAO ESP32-C3.

8.3.2 Programming the microcontroller

Upload both files from the project code folder to your microcontroller:

- `project_5_wifi_mood_lamp.py`
- `resources/project_5_wifi_mood_lamp.html`

Keep the `resources/` folder name exactly the same on the microcontroller filesystem. The Python script loads the dashboard from that path.

Then open `project_5_wifi_mood_lamp.py` in the IDE and run it. Refer back to Chapter 3 for IDE instructions. When the script starts, watch the serial output. It prints:

- A unique Wi-Fi network name (SSID), such as `MoodLamp-1A2B3C`
- The Wi-Fi password defined in code
- The local IP address for the control page

From your laptop or phone:

1. Open Wi-Fi settings and connect to the printed SSID
2. Enter the password
3. Open a browser and navigate to the printed IP address (for example `http://192.168.4.1/`)

The dashboard includes:

- **Global controls:** Set color and brightness for all LEDs at once
- **Per-LED controls:** Set each LED independently
- **Animation controls:** Start/stop breathing, rainbow, wipe, and blink

Tip: If many students are in the room, the unique suffix in the SSID helps you identify your own board quickly.

8.4 Review

In this project, you combined embedded networking and LED control in one program. You practiced:

- Hosting a Wi-Fi access point directly on an ESP32-C3
- Serving a local web interface from a microcontroller
- Parsing user commands from HTTP requests
- Driving WS2812B LEDs with both static colors and animated effects
- Designing a system that is interactive, wireless, and real-time

8.5 Possible Extensions

- Add a “favorites” list of saved color presets
- Add a timed auto-off feature (for example, sleep after 30 minutes)
- Add a “party mode” that randomly cycles between multiple effects
- Add an ambient light sensor and auto-adjust brightness
- Add a physical button that toggles between saved scenes without opening the web page

Chapter 9: Project 6: ESP-NOW Hot Potato

9.1 Overview

This project is a placeholder for Project 6 (ESP-NOW Hot Potato). A multiplayer wireless game where a countdown timer is passed between boards using the ESP-NOW protocol. The student holding the “potato” when the timer hits zero loses!

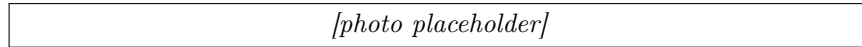


Figure 9.1: The end result should look something like this

9.2 Directions

9.2.1 Creating the circuit

9.2.2 Programming the microcontroller

Click on the file named "project_6_hot_potato.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

9.3 Review

9.4 Possible Extensions

Chapter 10: Project 7: Tilting Ball Maze

10.1 Overview

In this project, you will build a handheld, tilt-controlled arcade game: the **Tilting Ball Maze**! By wiring an MPU-6050 6-axis accelerometer and a 128×64 OLED display to your ESP32-C3 over a shared I²C bus, your microcontroller will measure how much you tilt the breadboard in real time and roll a ball sprite through a maze on screen. Guide the ball from the starting corner to the goal without getting trapped in dead ends!

Over the course of this project, you will:

- Connect multiple digital devices to a single shared I²C bus
- Read tilt and gravity data from an MPU-6050 accelerometer
- Drive a high-resolution monochrome OLED screen using MicroPython’s `framebuf` graphics library
- Implement 2D physics (acceleration, velocity, friction, and wall collisions) in Python
- Build a complete, responsive interactive game

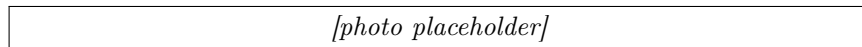


Figure 10.1: The end result should look something like this

Learning goals:

- Understand how the I²C protocol uses two shared wires (SCL and SDA) to talk to multiple chips
- Measure physical tilt using 16-bit accelerometer readings
- Draw shapes, text, and sprites on an SSD1306 OLED display
- Calculate axis-aligned bounding box (AABB) collisions between a moving ball and maze walls

10.2 Components

- Seeed XIAO ESP32-C3 microcontroller
- 0.96-inch 128×64 SSD1306 I²C OLED display
- MPU-6050 6-axis accelerometer/gyroscope module (GY-521)
- Breadboard
- Jumper wires

10.3 Understanding I²C Communication

Up to now, every peripheral had its own dedicated GPIO pin. However, microcontrollers have a limited number of pins. The **I²C** (Inter-Integrated Circuit) communication protocol solves this problem by allowing multiple sensor and display chips to share the exact same two wires:

- **SCL (Serial Clock):** A clock signal that synchronizes data transmission between chips.
- **SDA (Serial Data):** A bidirectional data line that carries the messages.

Every I²C device has a unique digital **address** (like a street address). When the microcontroller sends data on the SDA wire, it prefixes the message with the target device's address. All other devices on the bus ignore the message.

In this project, both the OLED display (address **0x3C**) and the MPU-6050 accelerometer (address **0x68**) connect to the same pair of microcontroller pins: **GPIO 7** (SCL) and **GPIO 6** (SDA).

Important: Power both the OLED and the MPU-6050 from **3.3V**, not 5V. The ESP32-C3 GPIO pins operate at 3.3V logic levels. Connecting 5V power to the sensors could damage the microcontroller.

10.4 Directions

Remove previous components

Before beginning, remove any components and wires from previous projects. Leave the microcontroller firmly seated in its standard position.

10.4.1 Creating the circuit

Attach the microcontroller to the breadboard

Orient the microcontroller so that the pin labeled **5V** goes into hole **H1** (Row H, Column 1), **GPIO2** into **D1**, **GPIO20** into **H7**, and **GPIO21** into **D7**. Then press it down—this might take more force than you expect.

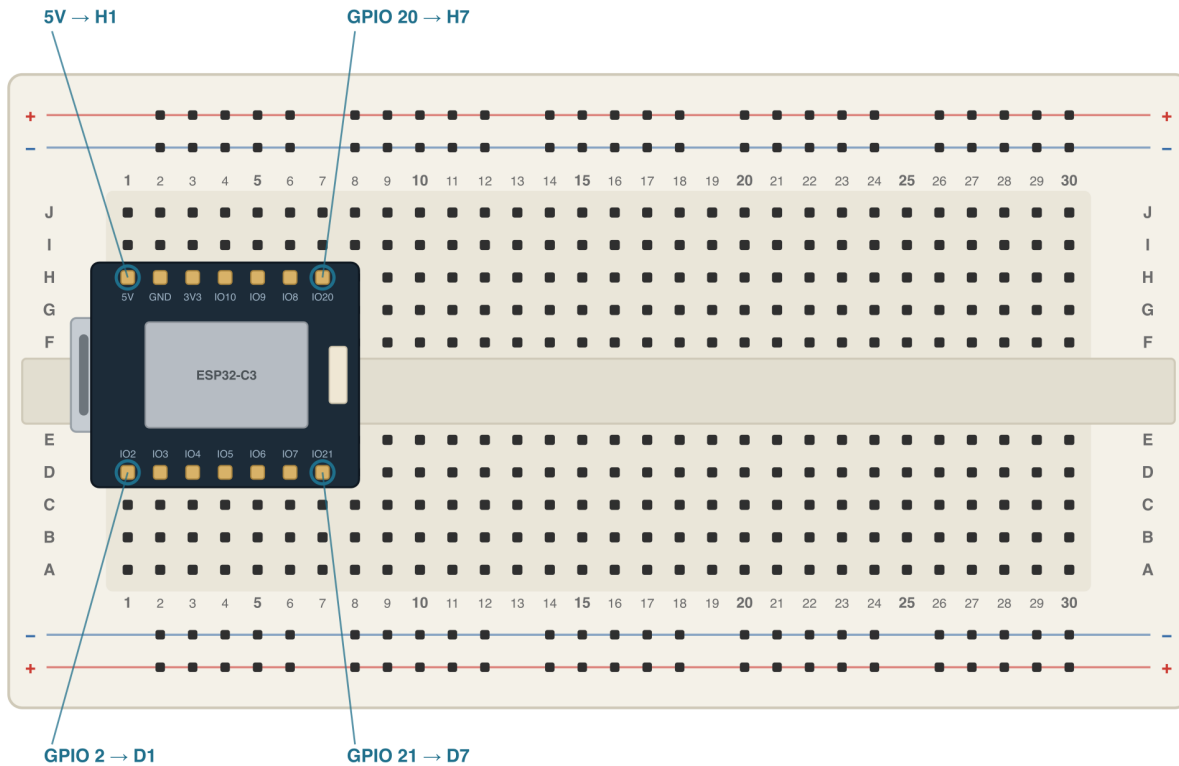


Figure 10.2: So far, so good!

MPU-6050 accelerometer placement and wiring

The MPU-6050 (GY-521) breakout board connects with straight-through header pins and sits flat facing upward. Push the module's eight pins into column **A** (rows 12–19) on the front of the breadboard: **INT** into **A12**, **AD0** into **A13**, **XCL** into **A14**, **XDA** into **A15**, **SDA** into **A16**, **SCL** into **A17**, **GND** into **A18**, and **VCC** into **A19**.

Only four pins (VCC, GND, SCL, SDA) are needed. Pins INT, AD0, XCL, and XDA remain unconnected. Connect the MPU-6050 jumpers into the corresponding holes in row **E**:

MPU-6050 pin	XIAO pin	Jumper holes
VCC	3V3	J3 → E19
GND	GND	J2 → E18
SCL	GPIO 7	B6 → E17
SDA	GPIO 6	B5 → E16

OLED display placement and wiring

The 0.96-inch SSD1306 OLED module also connects with straight-through header pins, sitting flat facing upward. Insert the four pins into column **A** (rows 25–28) next to the accelerometer: **GND** into **A25**, **VCC** into **A26**, **SCL** into **A27**, and **SDA** into **A28**.

Check your OLED pins: Most OLED displays have the pin order **GND**, **VCC**, **SCL**, **SDA**. However, some modules swap **VCC** and **GND**. Always check the silkscreen labels printed on your specific OLED board before powering on!

Connect the OLED by daisy-chaining power, ground, clock, and data directly from the accelerometer:

OLED pin	Connected to	Jumper holes
GND	MPU GND	D18 → E25
VCC	MPU VCC	D19 → E26
SCL	MPU SCL	D17 → E27
SDA	MPU SDA	D16 → E28

All four connections are daisy-chained through the accelerometer rows. Its **VCC** and **GND** connections continue from rows 19 and 18 to the OLED at rows 26 and 25. Likewise, **GPIO 7** connects to the accelerometer's SCL pin at row 17 and

continues to the OLED's SCL pin at row 27, while **GPIO 6** connects to the accelerometer's SDA pin at row 16 and continues to the OLED's SDA pin at row 28. Both devices therefore share the same power and I²C bus connections.

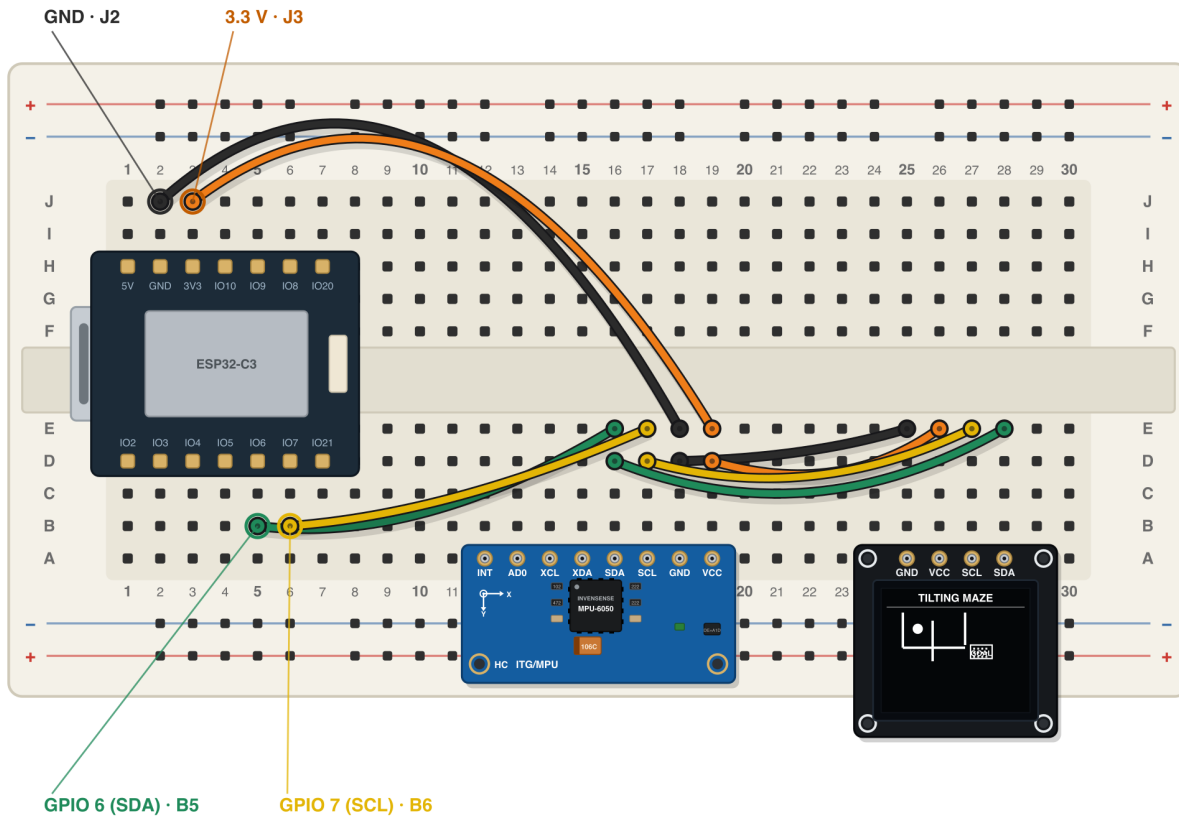


Figure 10.3: Connect both the OLED display and MPU-6050 accelerometer to the shared I2C bus.

10.4.2 Programming the microcontroller

Upload the following files to your microcontroller using the IDE:

- `project_7_tilting_ball_maze.py`
- `lib/ssd1306.py` (in the `lib/` directory)
- `lib/mpu6050.py` (in the `lib/` directory)

Once the files are uploaded, open `project_7_tilting_ball_maze.py` and click the play button. Refer back to Chapter 3 for IDE instructions.

10.4.3 How the program works

Initializing the Shared I²C Bus

The program begins by creating a hardware I²C bus on GPIO 7 (SCL) and GPIO 6 (SDA), then scans for connected devices:

Listing 10.1: Initializing I2C and discovering devices

```
1 i2c = I2C(0, scl=Pin(7), sda=Pin(6), freq=400000)
2 devices = i2c.scan()
3 print("I2C devices found:", [hex(d) for d in devices])
```

If both modules are wired correctly, the serial console prints `['0x3c', '0x68']`.

Reading Tilt from the Accelerometer

The MPU-6050 contains micro-electromechanical sensors (MEMS) that measure acceleration in three dimensions (X , Y , and Z). When the breadboard is held flat, gravity points straight down along the Z -axis. When you tilt the board, gravity tilts into the X and Y axes:

Listing 10.2: Converting tilt into ball acceleration

```
1 ax, ay, az = mpu.read_accel()
2
3 # Remove the neutral offsets measured while the board was level
4 ax -= ax_offset
5 ay -= ay_offset
6
7 # Apply a small deadzone so the ball rests still on flat surfaces
8 if abs(ax) < DEADZONE:
9     ax = 0
10 if abs(ay) < DEADZONE:
11     ay = 0
12
13 # Integrate acceleration into velocity with friction
14 vx = (vx + ax * TILT_SCALE) * FRICTION
15 vy = (vy + ay * TILT_SCALE) * FRICTION
```

Accelerometers usually report a small nonzero value even when perfectly level. When the program starts, it displays **Hold level...** and averages 100 readings to find that neutral X/Y offset. Keep the breadboard level and still until the game title appears. The program subtracts those offsets from every later reading so motion should be equally responsive in opposite directions.

Collision Detection and Drawing

The maze consists of rectangular wall segments. Before updating the ball's position, the game tests whether the new position would overlap any wall. If a collision is detected on an axis, the ball stops against the wall:

Listing 10.3: Axis-aligned bounding box collision

```
1 new_x = ball_x + vx
2 if not hits_wall(new_x, ball_y, BALL_RADIUS):
3     ball_x = new_x
4 else:
5     vx = 0 # Stop horizontal motion on impact
```

Every frame, the program clears the screen buffer, draws all maze walls, renders the checkered goal zone, draws the ball sprite, and calls `oled.show()` to push the entire image to the OLED over I²C.

If something does not work:

- **I²C scan is empty:** Check the power (3.3V) and ground jumpers. Verify that SCL is on GPIO 7 (**D17** and **B6**) and SDA is on GPIO 6 (**D16** and **B5**).
- **OLED remains dark:** Check whether your OLED module has GND and VCC in reversed order. Verify that `lib/ssd1306.py` was uploaded to the `lib/` folder.
- **Ball moves backwards:** Tilting forwards should roll the ball down; tilting right should roll it right. If an axis feels inverted, change `INVERT_X` or `INVERT_Y` from `False` to `True` at the top of the script.
- **Ball responds too slowly:** Increase `TILT_SCALE` slightly. If very small tilts are ignored, decrease `DEADZONE` slightly.
- **Ball drifts or responds differently in opposite directions:** Restart the program and keep the breadboard level and still during calibration. If it still drifts, slightly increase `DEADZONE`.

10.5 Review

In this project, you combined motion sensing and display hardware on a single bus. You learned:

- How the I²C protocol connects multiple sensors and displays to just two microcontroller pins
- How an accelerometer detects gravitational tilt
- How to use MicroPython framebuffer commands (`fill`, `rect`, `fill_rect`, `text`)
- How basic physics simulation (velocity, dampening friction, and bounding box collisions) creates fluid game movement

10.6 Possible Extensions

- **Design custom mazes:** Modify the `WALLS` list in the Python script to create your own challenging labyrinths.
- **Add hazard holes:** Add circular “pit” zones that send the ball back to the starting point if touched!
- **Add a timer:** Use `utime.ticks_ms()` to track how fast players solve the maze and display their best time.
- **Multi-level maze:** When the goal is reached, load a new, more difficult maze layout.
- **Add sound effects:** Reconnect the speaker from Projects 3 and 4 to play a click when bumping into walls and a fanfare on victory!

Appendix A: Python Primer

If you are new to programming or need a refresher, this primer covers the essential Python and MicroPython concepts used throughout this workshop.

The primer is split into two parts:

1. **General Python Fundamentals:** Core syntax, variables, functions, loops, and logic.
2. **MicroPython on Microcontrollers:** How Python controls hardware pins, reads sensors, runs continuous event loops, and uses helper classes.

MicroPython vs. Standard Python (CPython): MicroPython is a lightweight, efficient implementation of Python 3 optimized to run directly on microcontrollers like the ESP32-C3. It includes built-in modules like `machine` for controlling hardware (pins, PWM, analog inputs, and I²C).

While some large desktop libraries (such as `pandas` or `pygame`) are not available in MicroPython, core Python syntax, data structures, and control flow are identical. What you learn here applies directly to general Python programming!

A.1 Part 1: General Python Fundamentals

A.1.1 A simple Python script

Here is a small Python example demonstrating variables, functions, loops, and conditions:

Listing A.1: A basic Python script

```
1 def show_message(text, repeat=1):
2     """Prints a message to the console with iteration numbers."""
3     if repeat > 1:
4         print("Repeating", repeat, "times:")
5
6     for i in range(repeat):
7         print("Iteration", i, ":", text)
8
9 # Main program flow
10 user_name = "Emily"
11 show_message(user_name)
12 show_message(user_name, repeat=3)
```

A.1.2 Breaking down the syntax

- **Defining functions (def):** On line 1, `def show_message(text, repeat=1):` defines a reusable function. The parameter `text` is required, while `repeat=1` is an **optional parameter** with a default value of 1.
- **Docstrings and comments:** Line 2 is a docstring (`"""..."""`) describing what the function does. Lines starting with `#` (like line 9) are comments ignored by Python, written to help humans understand the code.
- **Indentation matters:** Python does not use curly brackets (`{ }`) to enclose code blocks. Instead, it uses **indentation** (typically 4 spaces). All lines indented under `def`, `if`, or `for` belong to that block.
- **Conditionals (if):** Line 3 checks if `repeat > 1`. If true, it executes the indented `print` statement.
- **Loops and range():** Line 6 starts a `for` loop. In Python, `range(3)` generates the sequence of numbers 0, 1, 2 (starting at 0 and running for 3 steps). The variable `i` takes on each value in turn.
- **Variables and function calls:** Line 10 assigns the string "Emily" to the variable `user_name`. Line 11 calls `show_message` with default repeat (1 time). Line 12 calls it with `repeat=3`, which prints 3 iterations.

Running this script produces the following console output:

```
Iteration 0 : Emily
Repeating 3 times:
Iteration 0 : Emily
Iteration 1 : Emily
Iteration 2 : Emily
```

A.2 Part 2: MicroPython on Hardware

A.2.1 The microcontroller program anatomy

Unlike a desktop script that runs once and exits, a microcontroller typically runs a continuous **event loop** to monitor sensors and update outputs.

Here is an excerpt illustrating the standard hardware pattern from Project 1:

Listing A.2: A typical MicroPython hardware script

```
1 import time
2 from machine import Pin, ADC, PWM
3
4 # Constants
5 ADC_MAX = 4095
6 PWM_MAX = 1023
7
8 # Hardware configuration
9 sensor = ADC(Pin(2))
10 sensor.atten(ADC.ATTN_11DB)
11 led = PWM(Pin(20), freq=1000)
12
13 try:
14     while True:
15         # Read the light sensor (0 to 4095)
16         level = sensor.read()
17
18         # Scale sensor reading to LED PWM duty (0 to 1023)
19         duty = (ADC_MAX - level) * PWM_MAX // ADC_MAX
20         led.duty(duty)
21
22         print("Sensor:", level, "PWM Duty:", duty)
23         time.sleep(0.05)
24
25 except KeyboardInterrupt:
26     # Clean up when the stop button is pressed in the IDE
27     led.duty(0)
28     led.deinit()
```

A.2.2 Key hardware programming patterns

- **The machine module:** Provides drivers for physical pins. `Pin(20)` creates a GPIO controller; wrapping it in `PWM(...)` enables pulse-width modulation for brightness or sound; wrapping it in `ADC(...)` enables analog voltage measurement.
- **The continuous while True: loop:** Microcontrollers do not have operating systems with windows and mice. The program enters a loop that repeats indefinitely, reading inputs and controlling outputs.
- **Integer division (//):** MicroPython uses `//` for integer division, which discards the remainder. This is useful for calculating whole-number PWM duty values or pixel positions.
- **Pacing with time.sleep():** A brief pause (such as 50 milliseconds) prevents the loop from consuming 100% of the CPU and keeps the console output readable.
- **Clean exits with try / except KeyboardInterrupt:** When you press the red Stop button in the Viper IDE, it sends a `KeyboardInterrupt` signal. Wrapping the main loop in `try / except` allows the program to turn off LEDs and release hardware resources cleanly.

A.2.3 Object-Oriented Programming (Classes) in MicroPython

As your projects grow, grouping data (state) and functions (behavior) into **classes** keeps your code organized. For example, the `DebounceButton` class used in Projects 2, 3, and 4 tracks pin state and debounce timers:

Listing A.3: A simple helper class for a pushbutton

```
1 class SimpleButton:
2     def __init__(self, pin_num, action):
3         """Initializes the pin with a pull-up resistor and stores a callback."""
4         self.pin = Pin(pin_num, mode=Pin.IN, pull=Pin.PULL_UP)
5         self.action = action
6         self.press_count = 0
7
8     def check(self):
9         """Checks if the button was pressed (LOW when using pull-up)."""
10        if self.pin.value() == 0:
11            self.press_count += 1
12            self.action(self.press_count)
```

- **__init__ method:** Automatically called when a new object is created (e.g., `btn = SimpleButton(21, on_press)`). It initializes variables unique to that instance.
- **self:** Refers to the specific object instance, allowing methods to access and modify its own properties (like `self.press_count`).
- **Callbacks (functions as arguments):** Passing a function to `action` lets the button trigger your custom logic whenever a press is detected.

A.3 MicroPython Concepts Reference Map

Here is a quick reference for Python features you will encounter in each project:

Concept	What it does	Where used
Tuples & Lists	Ordered collections like RGB colors (255, 0, 0) and indexable lists <code>pixels[0]</code>	Projects 2, 4, 5
Callbacks	Passing a function name as an argument to run on button events	Projects 2, 3, 4
Millisecond Timers	Measuring reaction times and delays using <code>time.ticks_ms()</code> and <code>time.ticks_diff()</code>	Projects 2, 5
State Variables	Using variables to track game phases (<code>STATE_RED</code> , <code>STATE_GREEN</code>)	Project 2
Hardware Interrupts	Rapidly detecting rotary encoder clicks without polling	Project 3
Dictionaries & JSON	Key-value data mapping (<code>{"mode": "rainbow"}</code>) for network APIs	Project 5
I ² C Communication	Communicating with multiple digital sensors over a shared 2-wire bus	Project 7

Table A.1: Python and MicroPython concepts used across the workshop projects.